In [126…
```
!jupyter nbconvert --to notebook --execute preprocess.ipynb --inplace
```

```
[NbConvertApp] Converting notebook preprocess.ipynb to notebook
^C
Traceback (most recent call last):
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbclient/client.py", line 606,
in setup_kernel
    yield
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/preprocessors/execut
e.py", line 103, in preprocess
    self.preprocess_cell(cell, resources, index)
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/preprocessors/execut
e.py", line 124, in preprocess_cell
    cell = self.execute_cell(cell, index, store_history=True)
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/jupyter_core/utils/__init__.p
y", line 165, in wrapped
    return loop.run_until_complete(inner)
           ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
  File "/usr/lib64/python3.14/asyncio/base_events.py", line 706, in run_until_complete
    self.run_forever()
    ~~~~~~~~~~~~~~~~^^
  File "/usr/lib64/python3.14/asyncio/base_events.py", line 677, in run_forever
    self._run_once()
    ~~~~~~~~~~~~~~^^
  File "/usr/lib64/python3.14/asyncio/base_events.py", line 2019, in _run_once
    event_list = self._selector.select(timeout)
  File "/usr/lib64/python3.14/selectors.py", line 452, in select
    fd_event_list = self._selector.poll(timeout, max_ev)
KeyboardInterrupt

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/home/pedrosalgado22/.local/bin/jupyter-nbconvert", line 8, in <module>
    sys.exit(main())
             ~~~~^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/jupyter_core/application.py", l
ine 284, in launch_instance
    super().launch_instance(argv=argv, **kwargs)
    ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/traitlets/config/application.p
y", line 1075, in launch_instance
    app.start()
    ~~~~~~~~~^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/nbconvertapp.py", lin
e 420, in start
    self.convert_notebooks()
    ~~~~~~~~~~~~~~~~~~~~~~^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/nbconvertapp.py", lin
e 597, in convert_notebooks
    self.convert_single_notebook(notebook_filename)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/nbconvertapp.py", lin
e 563, in convert_single_notebook
    output, resources = self.export_single_notebook(
                        ~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        notebook_filename, resources, input_buffer=input_buffer
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/nbconvertapp.py", lin
e 487, in export_single_notebook
    output, resources = self.exporter.from_filename(
                        ~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        notebook_filename, resources=resources
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/exporters/exporter.p
y", line 201, in from_filename
    return self.from_file(f, resources=resources, **kw)
           ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/exporters/exporter.p
y", line 220, in from_file
    return self.from_notebook_node(
           ~~~~~~~~~~~~~~~~~~~~~~~^
```

```
        nbformat.read(file_stream, as_version=4), resources=resources, **kw
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/exporters/notebook.py", line 36, in from_notebook_node
    nb_copy, resources = super().from_notebook_node(nb, resources, **kw)
                         ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/exporters/exporter.py", line 154, in from_notebook_node
    nb_copy, resources = self._preprocess(nb_copy, resources)
                         ~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/exporters/exporter.py", line 353, in _preprocess
    nbc, resc = preprocessor(nbc, resc)
                ~~~~~~~~~~~~^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/preprocessors/base.py", line 48, in __call__
    return self.preprocess(nb, resources)
           ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbconvert/preprocessors/execute.py", line 97, in preprocess
    with self.setup_kernel():
         ~~~~~~~~~~~~~~~~~^^
  File "/usr/lib64/python3.14/contextlib.py", line 162, in __exit__
    self.gen.throw(value)
    ~~~~~~~~~~~~~~^^^^^^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/nbclient/client.py", line 609, in setup_kernel
    self._cleanup_kernel()
    ~~~~~~~~~~~~~~~~~~~~^^
  File "/home/pedrosalgado22/.local/lib/python3.14/site-packages/jupyter_core/utils/__init__.py", line 165, in wrapped
    return loop.run_until_complete(inner)
           ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
  File "/usr/lib64/python3.14/asyncio/base_events.py", line 706, in run_until_complete
    self.run_forever()
    ~~~~~~~~~~~~~~~~^^
  File "/usr/lib64/python3.14/asyncio/base_events.py", line 677, in run_forever
    self._run_once()
    ~~~~~~~~~~~~~~^^
  File "/usr/lib64/python3.14/asyncio/base_events.py", line 2019, in _run_once
    event_list = self._selector.select(timeout)
  File "/usr/lib64/python3.14/selectors.py", line 452, in select
    fd_event_list = self._selector.poll(timeout, max_ev)
KeyboardInterrupt
```

In [126…

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from pulp import *
from scipy.stats import poisson, norm
from pulp import LpProblem, LpMaximize, LpVariable, LpStatus, lpSum, value, PULP_CBC_CMD
```

In [126…

```
OUT_CSV = "fantasy_enriched.csv"
```

Position	Point Source	Points	Description
All Players	Appearance (<60 min)	+1	Plays any minutes up to 59:59
All Players	Appearance (60+ min)	+1	Additional point for reaching 60+ minutes (total appearance = **2 points**)
All Players	Assist	+3	Provides an assist
All Players	Yellow Card	-1	Receives a yellow card
All Players	Red Card	-2	Receives a red card
All Players	Own Goal	-2	Scores an own goal
All Players	Winning a Penalty	+2	Wins a penalty for their team
All Players	Conceding a Penalty	-1	Commits a foul leading to a penalty

Position	Point Source	Points	Description

Position	Point Source	Points	Description
Goalkeeper	Clean Sheet	+5	Plays 60+ minutes and team keeps a clean sheet
Goalkeeper	First Goal Conceded	0	No deduction for conceding the first goal
Goalkeeper	Each Additional Goal Conceded	-1	Every goal conceded after the first
Goalkeeper	Goal Scored	+9	Scores a goal
Goalkeeper	Penalty Save	+3	Saves a penalty during normal play (not shootouts)
Goalkeeper	Every 3 Saves	+1	Earns 1 point for every 3 saves

Position	Point Source	Points	Description
Defender	Clean Sheet	+5	Plays 60+ minutes and team keeps a clean sheet
Defender	First Goal Conceded	0	No deduction for conceding the first goal
Defender	Each Additional Goal Conceded	-1	Every goal conceded after the first
Defender	Goal Scored	+7	Scores a goal

Position	Point Source	Points	Description
Midfielder	Clean Sheet	+1	Plays 60+ minutes and team keeps a clean sheet
Midfielder	Goal Scored	+6	Scores a goal
Midfielder	Every 3 Tackles	+1	Earns 1 point for every 3 tackles
Midfielder	Every 2 Big Chances Created	+1	Earns 1 point for every 2 big chances created

Position	Point Source	Points	Description
Forward	Goal Scored	+5	Scores a goal
Forward	Every 2 Shots on Target	+1	Earns 1 point for every 2 shots on target

Position	Point Source	Points	Description
Bonus (All Players)	Direct Free-Kick Goal	+1	Additional bonus for scoring directly from a free kick (on top of goal points)
Bonus (All Players)	Scouting Bonus	+2	Scores **more than 4 base fantasy points** in a match **and** is selected by **<5%** of fantasy teams. Captaincy and booster points do **not** count toward the 4-point threshold.

Core identity / player info

Column	Type	Description
`fifa_id`	int	Unique FIFA fantasy player ID
`name`	str	Player name
`squad_id`	int	Internal squad/nation ID
`team`	str	National team
`position`	str	DEF / MID / FWD / GK
`price`	float	Fantasy price
`status`	str	Availability status
`total_points`	int	Total tournament points
`avg_points`	float	Average points per match
`matches_played`	int	Matches with participation
`percent_selected`	float	Selection rate (%)
`next_fixture`	int	Next match ID

Round fantasy points

Column	Type	Description
round_1_points	int	Points in round 1
round_2_points	int	Points in round 2
round_3_points	int	Points in round 3
round_4_points	int	Points in round 4

Betting / scorer mapping

Column	Type	Description
betano_matched_name	str	Matched Betano market name
betano_match_score	float	Name matching confidence (0–100)
anytime_scorer_prob	float	Probability player scores anytime
first_scorer_prob	float	Probability scores first goal
last_scorer_prob	float	Probability scores last goal

Match context

Column	Type	Description
match_home	str	Home team
match_away	str	Away team
is_home	bool	Player team is home
opponent	str	Opponent team

Match probabilities

Column	Type	Description
match_home_win_prob	float	Home win probability
match_draw_prob	float	Draw probability
match_away_win_prob	float	Away win probability
match_btts_prob	float	Both teams score probability
match_over_25_prob	float	Over 2.5 goals probability
match_over_35_prob	float	Over 3.5 goals probability
team_score_prob	float	Team scores ≥1 goal probability
team_score_2_prob	float	Team scores ≥2 goals probability
team_cs_prob	float	Team clean sheet probability
opp_score_prob	float	Opponent scores ≥1 goal probability
opp_cs_prob	float	Opponent clean sheet probability
team_over_05_prob	float	Team scores ≥1 goal probability
team_over_15_prob	float	Team scores ≥2 goals probability
team_qualify_prob	float	Team advances probability
team_win2_prob	float	Team wins by 2+ goals probability
opp_over_05_prob	float	Opponent scores ≥1 goal probability

Raw cumulative stats

Column	Type	Description
`stat_GS`	int	Goals
`stat_AS`	int	Assists
`stat_CS`	int	Clean sheets
`stat_GC`	int	Goals conceded
`stat_MP`	int	Minutes played
`stat_YC`	int	Yellow cards
`stat_RC`	int	Red cards
`stat_ST`	int	Shots on target
`stat_SB`	int	Shots blocked
`stat_CC`	int	Chances created
`stat_PS`	int	Penalties saved
`stat_T`	int	Tackles
`stat_S`	int	Saves
`stat_SXI`	int	Starts (XI appearances)
`stat_OG`	int	Own goals
`stat_PC`	int	Penalties committed
`stat_PW`	int	Penalties won
`stat_FK`	int	Free kicks won

Round-by-round stats (1–4)

Pattern	Type	Description
`round_i_SXI`	int	Started (0/1)
`round_i_MP`	int	Minutes
`round_i_AS`	int	Assists
`round_i_YC`	int	Yellow cards
`round_i_RC`	int	Red cards
`round_i_OG`	int	Own goals
`round_i_PW`	int	Penalties won
`round_i_PC`	int	Penalties committed
`round_i_CS`	int	Clean sheets
`round_i_GS`	int	Goals
`round_i_GC`	int	Goals conceded
`round_i_PS`	int	Penalties saved
`round_i_T`	int	Tackles
`round_i_CC`	int	Chances created
`round_i_ST`	int	Shots on target
`round_i_FK`	int	Free kicks
`round_i_S`	int	Saves

Pattern	Type	Description
round_i_SB	int	Shots blocked

External matching / enrichment

Column	Type	Description
clubstats_matched_player	str	External player match
clubs_match_dist	float	Matching distance score

Form / usage trends

Column	Type	Description
started_last_match	int	Started last match
starts_last_3	int	Starts last 3 matches
starts_last_5	int	Starts last 5 matches
start_rate	float	Start frequency
minutes_last_3_avg	float	Avg minutes last 3
minutes_last_5_avg	float	Avg minutes last 5

Performance rates

Column	Type	Description
goal_rate	float	Goals per minute
goals_per_start	float	Goals per start
shots_on_target_per90	float	SOT per 90
goals_per_shot_on_target	float	Conversion rate
assist_rate	float	Assists per minute
chance_created_per90	float	Chances per 90
tackles_per90	float	Tackles per 90
cc_per90	float	Chances created per 90
sot_per90	float	Shots on target per 90
saves_per90	float	Saves per 90
yc_per90	float	Yellow cards per 90
rc_per90	float	Red cards per 90
penalty_conceded_rate	float	Penalties conceded per 90
own_goal_rate	float	Own goals per 90
penalties_won_per90	float	Penalties won per 90

Recent / streak features

Column	Type	Description
recent_goals_last3	int	Goals last 3 matches
goal_streak	int	Consecutive scoring matches
recent_assists_last3	int	Assists last 3 matches
recent_chances_created_per90	float	Recent creation rate

Column	Type	Description
recent_cs_rate	float	Clean sheet rate
recent_tackles_per90	float	Recent tackles
recent_cc_per90	float	Recent chances created
recent_sot_per90	float	Recent shots on target
recent_saves_per90	float	Recent saves
recent_penalties_won	int	Penalties won last 3

Expected value features

Column	Type	Description
goal_expectation	float	Scoring proxy
goal_expectation2	float	Scoring proxy (2+ goals)
assist_expectation	float	Assist proxy
cs_expectation	float	Clean sheet proxy
expected_minutes	float	Expected minutes
expected_gc_penalty	float	Defensive penalty
expected_tackle_points	float	Tackles contribution
expected_cc_points	float	Chance creation points
expected_sot_points	float	Shot contribution points
expected_save_points	float	Save contribution points

Fantasy points dynamics

Column	Type	Description
points_last1	int	Last match points
points_last3_avg	float	Avg last 3
points_last5_avg	float	Avg last 5
weighted_points	float	Recency weighted
rolling_std_points	float	Variance last 5
rolling_max_points	int	Max last 5

Team-position ranking features

Column	Type	Description
price_rank_team_position	float	Price rank
selected_rank_team_position	float	Selection rank
points_rank_team_position	float	Points rank
minutes_rank_team_position	float	Minutes rank
starts_rank_team_position	float	Starts rank
anytime_rank_team_position	float	Scoring odds rank
shots_rank_team_position	float	Shooting rank
cc_rank_team_position	float	Chance creation rank
tackles_rank_team_position	float	Tackles rank

In [126…
```
df= pd.read_csv("fantasy_enriched.csv")
```

In [126…
```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

In [126…
```
df = df[df["status"] == "playing"].copy()
```

In [127…
```
# probability of playing a minute

raw_any = (
    0.2* df["minutes_rank_team_position"]
  + 0.10 * df["stat_MP"]         # minutes needs to be very team stratified since while for other
  + 0.15 * df["starts_rank_team_position"]
  + 0.10 * df["price_rank_team_position"]     # for minutes its a team stratified signal.
  + 0.1 * df["selected_rank_team_position"]
  + 0.25 * df["minutes_last_3_avg"]
  + 0.1 * df["has_betano_match"] # having odd at all means likeliness of playing is higher.

)

df["prob_plays_any"] = sigmoid((raw_any - 0.5) * 6)
```

In [127…
```
df.loc[
    :,
    [
        "prob_plays_any",
        "name",
        "team",
        "position",
        "stat_MP",
        "minutes_rank_team_position",
        "starts_rank_team_position",
        "selected_rank_team_position",
        "price_rank_team_position",
        "anytime_rank_team_position",
    ],
].sort_values("prob_plays_any", ascending=False).head(30)
```

Out[127...

	prob_plays_any	name	team	position	stat_MP	minutes_rank_team_position	starts_rank_tea
10	0.937027	Emiliano Martínez	Argentina	GK	1.000000	1.000000	
211	0.931775	Gustavo Gómez	Paraguay	DEF	1.000000	1.000000	
394	0.931263	Achraf Hakimi	Morocco	DEF	1.000000	1.000000	
148	0.920195	Kylian Mbappé	France	FWD	0.900000	1.000000	
127	0.918872	Harry Kane	England	FWD	0.907692	1.000000	
86	0.915806	Luis Díaz	Colombia	MID	0.917949	1.000000	
350	0.915676	Cristiano Ronaldo	Portugal	FWD	0.900000	1.000000	
65	0.915617	Jonathan David	Canada	FWD	0.848718	1.000000	
285	0.913635	Breel Embolo	Switzerland	FWD	0.889744	1.000000	
48	0.912136	Vinícius Júnior	Brazil	MID	0.900000	1.000000	
41	0.911742	Gabriel Magalhães	Brazil	DEF	0.923077	0.937500	
277	0.911742	Manuel Akanji	Switzerland	DEF	0.923077	0.937500	
185	0.908473	Ismael Saibari	Morocco	MID	0.930769	0.888889	
5	0.905064	Lionel Messi	Argentina	FWD	0.820513	1.000000	
232	0.902548	Nuno Mendes	Portugal	DEF	0.874359	0.875000	
107	0.902362	Mohamed Hany	Egypt	DEF	1.000000	1.000000	
250	0.902235	Bruno Fernandes	Portugal	MID	0.853846	1.000000	
39	0.901551	Youri Tielemans	Belgium	MID	0.987179	1.000000	
272	0.901130	Granit Xhaka	Switzerland	MID	0.923077	1.000000	
359	0.900698	Mohamed Salah	Egypt	MID	0.866667	0.900000	
112	0.898316	Omar Marmoush	Egypt	FWD	0.810256	1.000000	
42	0.898220	Marquinhos	Brazil	DEF	0.923077	0.937500	
344	0.895959	Leandro Trossard	Belgium	MID	0.925641	0.909091	
379	0.894740	Alistair Johnston	Canada	DEF	0.923077	1.000000	
82	0.894740	Dávinson Sánchez	Colombia	DEF	0.923077	0.937500	
143	0.894634	Dayot Upamecano	France	DEF	0.887179	1.000000	
298	0.894506	Weston McKennie	USA	MID	0.907692	1.000000	
19	0.894497	Alexis Mac Allister	Argentina	MID	0.843590	1.000000	
124	0.893199	Ezri Konsa	England	DEF	0.923077	1.000000	

	prob_plays_any	name	team	position	stat_MP	minutes_rank_team_position	starts_rank_tea
257	0.891881	Marc Cucurella	Spain	DEF	0.923077	0.937500	

In [127…
```
# probability of starting / playing 60 minutes

raw_any = (
    0.15 * df["minutes_rank_team_position"]
  + 0.10 * df["starts_rank_team_position"]
  + 0.15 * df["stat_MP"]
  + 0.05 * df["selected_rank_team_position"]
  + 0.35 * df["minutes_last_3_avg"]
  + 0.15 * df["round_4_MP"]
  + 0.05 * df["has_betano_match"] # having odd at all means likeliness of playing is higher.

)

df["prob_plays_60min"] = sigmoid((raw_any - 0.5) * 6)
```

In [127…
```
cols = [
    "name",
    "team",
    "position",
    "prob_plays_60min",
    "minutes_rank_team_position",
    "starts_rank_team_position",
    "price_rank_team_position",
    "selected_rank_team_position",
    "minutes_last_3_avg",
    "round_4_MP",
    "anytime_rank_team_position",
]

df[cols].sort_values("prob_plays_60min", ascending=False).head(30)
```

Out[127...

	name	team	position	prob_plays_60min	minutes_rank_team_position	starts_rank_team_positio
10	Emiliano Martínez	Argentina	GK	0.942676	1.000000	0.66666
211	Gustavo Gómez	Paraguay	DEF	0.939509	1.000000	0.57142
394	Achraf Hakimi	Morocco	DEF	0.939204	1.000000	0.56250
107	Mohamed Hany	Egypt	DEF	0.936805	1.000000	0.56250
28	Brandon Mechele	Belgium	DEF	0.934625	1.000000	0.55555
39	Youri Tielemans	Belgium	MID	0.930263	1.000000	0.54545
360	Andrés Cubas	Paraguay	MID	0.927574	1.000000	0.55555
401	Neil El Aynaoui	Morocco	MID	0.926888	1.000000	0.55555
340	Thibaut Courtois	Belgium	GK	0.924142	1.000000	0.66666
393	Yassine Bounou	Morocco	GK	0.924142	1.000000	0.66666
227	Orlando Gill	Paraguay	GK	0.924142	1.000000	0.66666
111	Yasser Ibrahim	Egypt	DEF	0.923769	0.875000	0.56250
117	Mostafa Shobeir	Egypt	GK	0.922371	1.000000	0.62500
185	Ismael Saibari	Morocco	MID	0.915987	0.888889	0.55555
209	Matías Galarza	Paraguay	MID	0.909457	0.777778	0.55555
344	Leandro Trossard	Belgium	MID	0.907041	0.909091	0.54545
359	Mohamed Salah	Egypt	MID	0.904995	0.900000	0.55000
127	Harry Kane	England	FWD	0.901582	1.000000	0.62500
86	Luis Díaz	Colombia	MID	0.900602	1.000000	0.54545
5	Lionel Messi	Argentina	FWD	0.899661	1.000000	0.62500
379	Alistair Johnston	Canada	DEF	0.899531	1.000000	0.56250
65	Jonathan David	Canada	FWD	0.898334	1.000000	0.58333
100	Marwan Attia	Egypt	MID	0.898334	1.000000	0.55000
277	Manuel Akanji	Switzerland	DEF	0.898167	0.937500	0.56250
257	Marc Cucurella	Spain	DEF	0.898167	0.937500	0.56250
41	Gabriel Magalhães	Brazil	DEF	0.898167	0.937500	0.56250
272	Granit Xhaka	Switzerland	MID	0.898098	1.000000	0.55000
148	Kylian Mbappé	France	FWD	0.897936	1.000000	0.66666

	name	team	position	prob_plays_60min	minutes_rank_team_position	starts_rank_team_positio
19	Alexis Mac Allister	Argentina	MID	0.896554	1.000000	0.54545
124	Ezri Konsa	England	DEF	0.896207	1.000000	0.55555

There is a phenomenon where a player will be overvalued, such as Lukaku here for belgium. He has not played much, but since Belgium only has 3 forward and only him has significative minutes he ranks number 1 for all ranks in BEL,FWD. Need to include more general sums and beware with manual selection.

In [127…
```
df["lambda_goal"] = -np.log(
    (1 - df["anytime_scorer_prob"]).clip(lower=0.001)  # clip prevents log(0)
)

lambda_adj = (
    0.8 * df["lambda_goal"]                          # dominant: Betano's model
      + 0.05 * df["recent_goals_last3"]
      + 0.02 * df["team_score_2_prob"]
      + 0.05* df["stat_GS"]
      + 0.05 * df["stat_ST"]
      + 0.03 * df["price"]
  )

# expected_goals is λ (not a probability — can exceed 1 for elite strikers)
df["prob_scores"] = lambda_adj.clip(lower=0) * df["prob_plays_60min"] * 1.5
```

In [127…
```
cols = [
    "name",
    "team",
    "position",
    "prob_scores",
    "goal_expectation",
    "anytime_scorer_prob",
    "recent_goals_last3",
    "recent_sot_per90",
    "goal_rate",
    "shots_on_target_per90",
    "stat_GS",
    "stat_ST",
    "price_rank_team_position",
]

df[cols].sort_values("prob_scores", ascending=False).head(30)
```

Out[127...

	name	team	position	prob_scores	goal_expectation	anytime_scorer_prob	recent_goals_last3
148	Kylian Mbappé	France	FWD	1.386756	0.558959	0.6536	1.00
5	Lionel Messi	Argentina	FWD	1.294161	0.499396	0.6173	1.00
261	Mikel Oyarzabal	Spain	FWD	0.867633	0.381892	0.4950	1.00
48	Vinícius Júnior	Brazil	MID	0.802062	0.340448	0.4348	0.75
149	Ousmane Dembélé	France	MID	0.771137	0.397754	0.4651	1.00
198	Erling Haaland	Norway	FWD	0.732862	0.285884	0.4405	0.75
127	Harry Kane	England	FWD	0.711353	0.256655	0.3817	0.75
350	Cristiano Ronaldo	Portugal	FWD	0.689669	0.254538	0.3922	0.75
185	Ismael Saibari	Morocco	MID	0.625631	0.279302	0.3745	0.50
31	Romelu Lukaku	Belgium	FWD	0.606995	0.335493	0.4587	0.50
318	Lautaro Martínez	Argentina	FWD	0.577739	0.371088	0.4587	0.25
49	Matheus Cunha	Brazil	FWD	0.572335	0.295504	0.3774	0.75
259	Lamine Yamal	Spain	MID	0.560119	0.302582	0.3922	0.25
161	Michael Olise	France	MID	0.523631	0.305392	0.3571	0.00
86	Luis Díaz	Colombia	MID	0.518737	0.238792	0.3390	0.00
285	Breel Embolo	Switzerland	FWD	0.499535	0.197463	0.3226	0.25
331	Bradley Barcola	France	MID	0.482283	0.326430	0.3817	0.25
6	Julián Alvarez	Argentina	FWD	0.475014	0.371088	0.4587	0.00
65	Jonathan David	Canada	FWD	0.471494	0.150419	0.2667	0.75
344	Leandro Trossard	Belgium	MID	0.445747	0.200404	0.2740	0.50
88	Luis Suárez	Colombia	FWD	0.406208	0.238792	0.3390	0.00
169	Julián Quiñones	Mexico	FWD	0.399127	0.156725	0.2500	0.50
313	Christian Pulisic	USA	MID	0.382460	0.279566	0.3846	0.00
39	Youri Tielemans	Belgium	MID	0.366470	0.157324	0.2151	0.50
40	Kevin De Bruyne	Belgium	MID	0.366189	0.200404	0.2740	0.25
402	Brahim Díaz	Morocco	MID	0.359969	0.216207	0.2899	0.00
140	Jude Bellingham	England	MID	0.355657	0.149407	0.2222	0.25
359	Mohamed Salah	Egypt	MID	0.350769	0.096718	0.2128	0.25

	name	team	position	prob_scores	goal_expectation	anytime_scorer_prob	recent_goals_last3
394	Achraf Hakimi	Morocco	DEF	0.348965	0.146251	0.1961	0.25
298	Weston McKennie	USA	MID	0.346898	0.186377	0.2564	0.00

In [127…
```
df["team_component"] = df["team_score_2_prob"] * df["prob_scores"]
prob_assists = (
    0.5 * df["chance_created_per90"]
  + 0.2 * df["team_component"]
  + 0.15 * df["assist_rate"]
  + 0.15 * df["stat_AS"]
)

df["lambda_assist"] = -np.log(
    (1 - prob_assists).clip(lower=0.001)
)

df["expected_assists"] = df["lambda_assist"].clip(lower=0) * df["prob_plays_60min"] * 2
```

In [127…
```
cols = [
    "name",
    "team",
    "position",
    "expected_assists",
    "assist_expectation",
    "assist_rate",
    "chance_created_per90",
    "team_score_2_prob",
    "recent_assists_last3",
    "team_component",
    "prob_plays_60min",
    "stat_AS",
]

df[cols].sort_values("expected_assists", ascending=False).head(30)
```

Out[127...

	name	team	position	expected_assists	assist_expectation	assist_rate	chance_created_per90
161	Michael Olise	France	MID	0.691256	1.249481	0.162338	0.162338
148	Kylian Mbappé	France	FWD	0.626543	0.219282	0.056980	0.028490
5	Lionel Messi	Argentina	FWD	0.469108	0.910125	0.000000	0.125000
321	Bruno Guimarães	Brazil	MID	0.395779	0.621794	0.117647	0.088235
327	Cucho Hernández	Colombia	FWD	0.379477	6.339600	1.000000	1.000000
149	Ousmane Dembélé	France	MID	0.368998	0.267250	0.069444	0.034722
171	Roberto Alvarado	Mexico	FWD	0.335359	0.663776	0.088235	0.117647
257	Marc Cucurella	Spain	DEF	0.320491	0.578625	0.083333	0.083333
285	Breel Embolo	Switzerland	FWD	0.278941	0.476274	0.057637	0.086455
48	Vinícius Júnior	Brazil	MID	0.273662	0.200769	0.028490	0.028490
261	Mikel Oyarzabal	Spain	FWD	0.271766	0.230681	0.033223	0.033223
369	Martin Ødegaard	Norway	MID	0.269062	0.447586	0.114943	0.076628
271	Johan Manzambi	Switzerland	MID	0.256088	0.826335	0.100000	0.150000
189	Patrick Berg	Norway	MID	0.245356	0.748846	0.085470	0.128205
402	Brahim Díaz	Morocco	MID	0.242329	0.450483	0.067114	0.067114
19	Alexis Mac Allister	Argentina	MID	0.235163	0.221307	0.060790	0.030395
75	Nathan Saliba	Canada	MID	0.209218	0.557802	0.109890	0.109890
331	Bradley Barcola	France	MID	0.204176	0.370038	0.048077	0.048077
140	Jude Bellingham	England	MID	0.202413	0.578178	0.031847	0.095541
344	Leandro Trossard	Belgium	MID	0.201934	0.364687	0.027701	0.055402
53	Rayan	Brazil	FWD	0.199449	0.634865	0.045045	0.090090
229	Julio Enciso	Paraguay	MID	0.187872	0.227706	0.061350	0.061350
394	Achraf Hakimi	Morocco	DEF	0.186963	0.344215	0.025641	0.051282
359	Mohamed Salah	Egypt	MID	0.186081	0.121021	0.059172	0.029586
198	Erling Haaland	Norway	FWD	0.184466	0.216333	0.037037	0.037037
313	Christian Pulisic	USA	MID	0.178060	0.792982	0.060606	0.121212
86	Luis Díaz	Colombia	MID	0.177524	0.177084	0.027933	0.027933
185	Ismael Saibari	Morocco	MID	0.174284	0.369818	0.000000	0.055096

	name	team	position	expected_assists	assist_expectation	assist_rate	chance_created_per90
139	Declan Rice	England	MID	0.169416	0.723299	0.039841	0.119522
293	Rubén Vargas	Switzerland	MID	0.165548	0.221241	0.080321	0.040161

In [127…
```
position_yc_modifier = {
    "GK": 0.15,
    "FWD": 0.35,
    "DEF": 0.45,
    "MID": 0.55,
}

df["position_yc_modifier"] = df["position"].map(position_yc_modifier)

raw_yc = (
    0.4 * df["yc_per90"]
  + 0.25 * df["tackles_per90"]
  + 0.20 * df["opp_over_05_prob"]
  + 0.05 * df["rc_per90"]
  + 0.1 * df["position_yc_modifier"]
)

df["prob_yellow_card"] = raw_yc * df["prob_plays_60min"]
```

In [127…
```
cols = [
    "name",
    "team",
    "position",
    "prob_yellow_card",
    "yc_per90",
    "stat_YC",
    "tackles_per90",
    "recent_tackles_per90",
    "opp_over_05_prob",
    "prob_plays_60min",
    "rc_per90",
]

df[cols].sort_values("prob_yellow_card", ascending=False).head(30)
```

Out[127...

	name	team	position	prob_yellow_card	yc_per90	stat_YC	tackles_per90	recent_tackles_per
209	Matías Galarza	Paraguay	MID	0.335370	0.255034	1.0	0.046980	0.2348
111	Yasser Ibrahim	Egypt	DEF	0.286044	0.202128	1.0	0.018617	0.1048
360	Andrés Cubas	Paraguay	MID	0.281007	0.097436	0.5	0.035897	0.1500
216	Juan José Cáceres	Paraguay	DEF	0.267306	0.108883	0.5	0.051576	0.2407
100	Marwan Attia	Egypt	MID	0.253950	0.110465	0.5	0.017442	0.0590
212	Júnior Alonso	Paraguay	DEF	0.242757	0.126667	0.5	0.026667	0.1428
211	Gustavo Gómez	Paraguay	DEF	0.232589	0.000000	0.0	0.010256	0.0500
59	Casemiro	Brazil	MID	0.228249	0.262976	1.0	0.041522	0.2254
189	Patrick Berg	Norway	MID	0.228009	0.162393	0.5	0.017094	0.0666
218	Miguel Almirón	Paraguay	MID	0.226822	0.177570	0.5	0.032710	0.1851
347	Rúben Dias	Portugal	DEF	0.225835	0.140741	0.5	0.018519	0.0925
236	Renato Veiga	Portugal	DEF	0.221707	0.105556	0.5	0.008333	0.0185
107	Mohamed Hany	Egypt	DEF	0.218386	0.000000	0.0	0.035897	0.1333
379	Alistair Johnston	Canada	DEF	0.213096	0.105556	0.5	0.002778	0.0000
22	Timothy Castagne	Belgium	DEF	0.212992	0.127517	0.5	0.033557	0.1239
359	Mohamed Salah	Egypt	MID	0.212568	0.000000	0.0	0.002959	0.0000
229	Julio Enciso	Paraguay	MID	0.212075	0.000000	0.0	0.012270	0.0423
28	Brandon Mechele	Belgium	DEF	0.211951	0.097436	0.5	0.007692	0.0166
44	Danilo	Brazil	DEF	0.209254	0.241270	1.0	0.006349	0.0370
75	Nathan Saliba	Canada	MID	0.208410	0.208791	0.5	0.032967	0.1648
272	Granit Xhaka	Switzerland	MID	0.207787	0.105556	0.5	0.013889	0.0925
103	Emam Ashour	Egypt	MID	0.207659	0.000000	0.0	0.009346	0.0400
21	Maxim De Cuyper	Belgium	DEF	0.203296	0.130137	0.5	0.010274	0.0387
227	Orlando Gill	Paraguay	GK	0.198690	0.000000	0.0	0.000000	0.0000
62	Derek Cornelius	Canada	DEF	0.198446	0.120635	0.5	0.015873	0.0222
409	Tyler Adams	USA	MID	0.197659	0.140741	0.5	0.025926	0.1388
279	Nico Elvedi	Switzerland	DEF	0.195859	0.105556	0.5	0.008333	0.0555
108	Ramy Rabia	Egypt	DEF	0.195492	0.000000	0.0	0.014388	0.0772

	name	team	position	prob_yellow_card	yc_per90	stat_YC	tackles_per90	recent_tackles_per
230	Diego Gómez	Paraguay	MID	0.194831	0.322034	1.0	0.033898	0.1923
112	Omar Marmoush	Egypt	FWD	0.190799	0.000000	0.0	0.012658	0.0663

In [128…
```
raw_pw = (
    0.10 * df["stat_PW"]
  + 0.3 * df["anytime_scorer_prob"]
  + 0.10 * df["chance_created_per90"]
  + 0.2 * df["stat_CC"]                  # recent form in
  + 0.2 * df["team_score_2_prob"]        # team expected to attack heavily
  + 0.10 * df["price"] # quality proxy
)

df["prob_pen_won"] = 0.2 * sigmoid((raw_pw - 0.65) * 5) * df["prob_plays_60min"]
```

In [128…
```
cols = [
    "name",
    "team",
    "position",
    "prob_pen_won",
    "stat_PW",
    "anytime_scorer_prob",
    "assist_expectation",
    "chance_created_per90",
    "team_score_2_prob",
    "price_rank_team_position",
    "prob_plays_60min",
]

df[cols].sort_values("prob_pen_won", ascending=False).head(30)
```

Out[128...

	name	team	position	prob_pen_won	stat_PW	anytime_scorer_prob	assist_expectation	char
5	Lionel Messi	Argentina	FWD	0.074171	0.0	0.6173	0.910125	
161	Michael Olise	France	MID	0.067358	0.0	0.3571	1.249481	
148	Kylian Mbappé	France	FWD	0.055986	0.0	0.6536	0.219282	
318	Lautaro Martínez	Argentina	FWD	0.042434	1.0	0.4587	0.299630	
149	Ousmane Dembélé	France	MID	0.040916	0.0	0.4651	0.267250	
48	Vinícius Júnior	Brazil	MID	0.038267	0.0	0.4348	0.200769	
394	Achraf Hakimi	Morocco	DEF	0.033569	0.0	0.1961	0.344215	
261	Mikel Oyarzabal	Spain	FWD	0.033030	0.0	0.4950	0.230681	
185	Ismael Saibari	Morocco	MID	0.032569	0.0	0.3745	0.369818	
39	Youri Tielemans	Belgium	MID	0.032234	1.0	0.2151	0.170977	
285	Breel Embolo	Switzerland	FWD	0.031905	0.0	0.3226	0.476274	
140	Jude Bellingham	England	MID	0.031024	0.0	0.2222	0.578178	
257	Marc Cucurella	Spain	DEF	0.030921	0.0	0.1081	0.578625	
232	Nuno Mendes	Portugal	DEF	0.030618	0.0	0.1333	0.513871	
132	Noni Madueke	England	FWD	0.029860	1.0	0.1818	0.747111	
330	Jules Koundé	France	DEF	0.029613	0.0	0.0833	0.459510	
198	Erling Haaland	Norway	FWD	0.029382	0.0	0.4405	0.216333	
321	Bruno Guimarães	Brazil	MID	0.029085	0.0	0.1250	0.621794	
331	Bradley Barcola	France	MID	0.028725	0.0	0.3817	0.370038	
344	Leandro Trossard	Belgium	MID	0.027630	0.0	0.2740	0.364687	
171	Roberto Alvarado	Mexico	FWD	0.026761	0.0	0.1667	0.663776	
86	Luis Díaz	Colombia	MID	0.026648	0.0	0.3390	0.177084	
127	Harry Kane	England	FWD	0.026260	0.0	0.3817	0.000000	
259	Lamine Yamal	Spain	MID	0.026053	0.0	0.3922	0.000000	
143	Dayot Upamecano	France	DEF	0.025998	0.0	0.1000	0.222451	
250	Bruno Fernandes	Portugal	MID	0.025324	0.0	0.2151	0.350811	
402	Brahim Díaz	Morocco	MID	0.025098	0.0	0.2899	0.450483	
350	Cristiano Ronaldo	Portugal	FWD	0.024798	0.0	0.3922	0.000000	
53	Rayan	Brazil	FWD	0.024441	0.0	0.2857	0.634865	

	name	team	position	prob_pen_won	stat_PW	anytime_scorer_prob	assist_expectation	char
19	Alexis Mac Allister	Argentina	MID	0.024153	0.0	0.2151	0.221307	

In [128…

```
raw_cs = (
    0.90 * df["team_cs_prob"]            # strongest signal
  + 0.05 * df["stat_CS"]                # tournament history
  + 0.05 * (1 - df["stat_GC"])          # fewer goals conceded is better
)

base_cs = sigmoid((raw_cs - 0.5) * 6)

df["prob_clean_sheet"] = base_cs * df["prob_plays_60min"]

df.loc[df["position"] == "FWD", "prob_clean_sheet"] = 0.0
```

In [128…

```
cols = [
    "name",
    "team",
    "position",
    "prob_clean_sheet",
    "team_cs_prob",
    "prob_plays_60min",
    "stat_CS",
    "stat_GC",
    "minutes_rank_team_position",
    "tackles_per90",
]

df[cols].sort_values("prob_clean_sheet", ascending=False).head(30)
```

Out[128...

	name	team	position	prob_clean_sheet	team_cs_prob	prob_plays_60min	stat_CS	stat_GC
143	Dayot Upamecano	France	DEF	0.559971	0.5876	0.887167	0.50	0.285714
152	Mike Maignan	France	GK	0.555670	0.5876	0.880352	0.50	0.285714
330	Jules Koundé	France	DEF	0.534562	0.5876	0.846911	0.50	0.285714
10	Emiliano Martínez	Argentina	GK	0.533920	0.5455	0.942676	0.50	0.428571
161	Michael Olise	France	MID	0.533820	0.5876	0.845736	0.50	0.285714
19	Alexis Mac Allister	Argentina	MID	0.533517	0.5455	0.896554	0.75	0.285714
149	Ousmane Dembélé	France	MID	0.527830	0.5876	0.801957	0.75	0.142857
314	Lisandro Martínez	Argentina	DEF	0.499077	0.5455	0.865129	0.50	0.285714
20	Enzo Fernández	Argentina	MID	0.493476	0.5455	0.855420	0.50	0.285714
329	William Saliba	France	DEF	0.491727	0.5876	0.766993	0.50	0.142857
155	Adrien Rabiot	France	MID	0.464984	0.5876	0.725280	0.50	0.142857
331	Bradley Barcola	France	MID	0.463736	0.5876	0.704575	0.75	0.142857
0	Cristian Romero	Argentina	DEF	0.449612	0.5455	0.779383	0.50	0.285714
315	Facundo Medina	Argentina	DEF	0.445140	0.5455	0.757934	0.50	0.142857
2	Nahuel Molina	Argentina	DEF	0.429082	0.5455	0.743795	0.50	0.285714
14	Rodrigo De Paul	Argentina	MID	0.427781	0.5455	0.728377	0.50	0.142857
159	Aurélien Tchouaméni	France	MID	0.414146	0.5876	0.674982	0.25	0.285714
146	Lucas Digne	France	DEF	0.395201	0.5876	0.607153	0.50	0.000000
18	Thiago Almada	Argentina	MID	0.378141	0.5455	0.624656	0.75	0.142857
394	Achraf Hakimi	Morocco	DEF	0.367471	0.4360	0.939204	0.25	0.571429
401	Neil El Aynaoui	Morocco	MID	0.362652	0.4360	0.926888	0.25	0.571429
393	Yassine Bounou	Morocco	GK	0.361578	0.4360	0.924142	0.25	0.571429
185	Ismael Saibari	Morocco	MID	0.358387	0.4360	0.915987	0.25	0.571429
86	Luis Díaz	Colombia	MID	0.356415	0.3879	0.900602	0.75	0.142857
396	Noussair Mazraoui	Morocco	DEF	0.355634	0.4360	0.826803	0.50	0.285714
82	Dávinson Sánchez	Colombia	DEF	0.353933	0.3879	0.894331	0.75	0.142857
81	Jhon Lucumí	Colombia	DEF	0.350746	0.3879	0.886278	0.75	0.142857
90	Camilo Vargas	Colombia	GK	0.348401	0.3879	0.880352	0.75	0.142857

	name	team	position	prob_clean_sheet	team_cs_prob	prob_plays_60min	stat_CS	stat_GC
94	Gustavo Puerta	Colombia	MID	0.347237	0.3879	0.877412	0.75	0.142857
257	Marc Cucurella	Spain	DEF	0.338124	0.3510	0.898167	1.00	0.000000

In [128…

```
position_rc_modifier = {
    "GK": 0.04,
    "FWD": 0.08,
    "DEF": 0.12,
    "MID": 0.14,
}

df["position_rc_modifier"] = df["position"].map(position_rc_modifier)

raw_rc = (
    0.10 * df["position_rc_modifier"]   # strongest prior
  + 0.25 * df["rc_per90"]               # direct history
  + 0.15 * df["tackles_per90"]          # physical involvement
  + 0.25 * df["prob_yellow_card"]       # disciplinary tendency
  + 0.25 * df["opp_over_05_prob"]       # more defending -> more challenges
)

df["prob_red_card"] = 0.1 * sigmoid((raw_rc - 0.5) * 5) * df["prob_plays_60min"]
```

In [128…

```
cols = [
    "name",
    "team",
    "position",
    "prob_red_card",
    "position_rc_modifier",
    "rc_per90",
    "tackles_per90",
    "prob_yellow_card",
    "opp_over_05_prob",
    "prob_plays_any",
]

df[cols].sort_values("prob_red_card", ascending=False).head(30)
```

Out[128...

	name	team	position	prob_red_card	position_rc_modifier	rc_per90	tackles_per90	prob_yellow
218	Miguel Almirón	Paraguay	MID	0.031198	0.14	0.728972	0.032710	0.2
209	Matías Galarza	Paraguay	MID	0.029664	0.14	0.000000	0.046980	0.3
360	Andrés Cubas	Paraguay	MID	0.028721	0.14	0.000000	0.035897	0.2
211	Gustavo Gómez	Paraguay	DEF	0.027320	0.12	0.000000	0.010256	0.2
216	Juan José Cáceres	Paraguay	DEF	0.027166	0.12	0.000000	0.051576	0.2
111	Yasser Ibrahim	Egypt	DEF	0.025784	0.12	0.000000	0.018617	0.2
227	Orlando Gill	Paraguay	GK	0.025189	0.04	0.000000	0.000000	0.1
107	Mohamed Hany	Egypt	DEF	0.024820	0.12	0.000000	0.035897	0.2
100	Marwan Attia	Egypt	MID	0.024518	0.14	0.000000	0.017442	0.2
212	Júnior Alonso	Paraguay	DEF	0.023765	0.12	0.000000	0.026667	0.2
229	Julio Enciso	Paraguay	MID	0.023657	0.14	0.000000	0.012270	0.2
359	Mohamed Salah	Egypt	MID	0.023592	0.14	0.000000	0.002959	0.2
103	Emam Ashour	Egypt	MID	0.022868	0.14	0.000000	0.009346	0.2
117	Mostafa Shobeir	Egypt	GK	0.022410	0.04	0.000000	0.000000	0.1
112	Omar Marmoush	Egypt	FWD	0.022053	0.08	0.000000	0.012658	0.1
108	Ramy Rabia	Egypt	DEF	0.022002	0.12	0.000000	0.014388	0.1
164	César Montes	Mexico	DEF	0.021362	0.12	0.577778	0.007407	0.1
236	Renato Veiga	Portugal	DEF	0.021286	0.12	0.000000	0.008333	0.2
232	Nuno Mendes	Portugal	DEF	0.020645	0.12	0.000000	0.014663	0.1
347	Rúben Dias	Portugal	DEF	0.020528	0.12	0.000000	0.018519	0.2
250	Bruno Fernandes	Portugal	MID	0.020202	0.14	0.000000	0.024024	0.1
28	Brandon Mechele	Belgium	DEF	0.020193	0.12	0.000000	0.007692	0.2
379	Alistair Johnston	Canada	DEF	0.020182	0.12	0.000000	0.002778	0.2
350	Cristiano Ronaldo	Portugal	FWD	0.020102	0.08	0.000000	0.005698	0.1
357	Mostafa Zico	Egypt	MID	0.019876	0.14	0.000000	0.016949	0.1
39	Youri Tielemans	Belgium	MID	0.019753	0.14	0.000000	0.010390	0.1
189	Patrick Berg	Norway	MID	0.019709	0.14	0.000000	0.017094	0.2

	name	team	position	prob_red_card	position_rc_modifier	rc_per90	tackles_per90	prob_yellow
249	João Neves	Portugal	MID	0.019344	0.14	0.000000	0.029900	0.18
298	Weston McKennie	USA	MID	0.019243	0.14	0.000000	0.022599	0.18
344	Leandro Trossard	Belgium	MID	0.019226	0.14	0.000000	0.013850	0.1

In [128…

```
position_og_modifier = {
    "GK": 0.05,
    "FWD": 0.02,
    "MID": 0.04,
    "DEF": 0.08,
}

df["position_og_modifier"] = df["position"].map(position_og_modifier)

raw_og = (
    0.4 * df["position_og_modifier"]
  + 0.5 * df["opp_over_05_prob"]
  + 0.10 * (1 - df["stat_GC"])
)

df["prob_own_goal"] = 0.03 * sigmoid((raw_og - 0.5) * 5) * df["prob_plays_60min"]
```

In [128…

```
cols = [
    "name",
    "team",
    "position",
    "prob_own_goal",
    "position_og_modifier",
    "own_goal_rate",
    "opp_over_05_prob",
    "tackles_per90",
    "stat_GC",
    "prob_plays_any",
]

df[cols].sort_values("prob_own_goal", ascending=False).head(30)
```

Out[128...

	name	team	position	prob_own_goal	position_og_modifier	own_goal_rate	opp_over_05_prob	ta
209	Matías Galarza	Paraguay	MID	0.017038	0.04	0.000000	1.000000	
211	Gustavo Gómez	Paraguay	DEF	0.016210	0.08	0.000000	1.000000	
216	Juan José Cáceres	Paraguay	DEF	0.015762	0.08	0.000000	1.000000	
227	Orlando Gill	Paraguay	GK	0.015537	0.05	0.000000	1.000000	
360	Andrés Cubas	Paraguay	MID	0.015458	0.04	0.000000	1.000000	
111	Yasser Ibrahim	Egypt	DEF	0.015135	0.08	0.000000	0.895719	
107	Mohamed Hany	Egypt	DEF	0.014849	0.08	0.005128	0.895719	
108	Ramy Rabia	Egypt	DEF	0.014518	0.08	0.000000	0.895719	
117	Mostafa Shobeir	Egypt	GK	0.014206	0.05	0.000000	0.895719	
100	Marwan Attia	Egypt	MID	0.014182	0.04	0.000000	0.895719	
212	Júnior Alonso	Paraguay	DEF	0.013854	0.08	0.000000	1.000000	
359	Mohamed Salah	Egypt	MID	0.013803	0.04	0.000000	0.895719	
229	Julio Enciso	Paraguay	MID	0.013695	0.04	0.000000	1.000000	
112	Omar Marmoush	Egypt	FWD	0.013598	0.02	0.000000	0.895719	
232	Nuno Mendes	Portugal	DEF	0.013469	0.08	0.000000	0.802133	
236	Renato Veiga	Portugal	DEF	0.013466	0.08	0.000000	0.802133	
103	Emam Ashour	Egypt	MID	0.013393	0.04	0.000000	0.895719	
347	Rúben Dias	Portugal	DEF	0.013315	0.08	0.000000	0.802133	
346	Diogo Costa	Portugal	GK	0.012958	0.05	0.000000	0.802133	
350	Cristiano Ronaldo	Portugal	FWD	0.012719	0.02	0.000000	0.802133	
250	Bruno Fernandes	Portugal	MID	0.012452	0.04	0.000000	0.802133	
379	Alistair Johnston	Canada	DEF	0.012201	0.08	0.000000	0.744899	
249	João Neves	Portugal	MID	0.011955	0.04	0.000000	0.802133	
61	Richie Laryea	Canada	DEF	0.011878	0.08	0.000000	0.744899	
357	Mostafa Zico	Egypt	MID	0.011837	0.04	0.000000	0.895719	
382	Stephen Eustaquio	Canada	MID	0.011772	0.04	0.000000	0.744899	
70	Maxime Crépeau	Canada	GK	0.011550	0.05	0.000000	0.744899	

	name	team	position	prob_own_goal	position_og_modifier	own_goal_rate	opp_over_05_prob	ta
28	Brandon Mechele	Belgium	DEF	0.011491	0.08	0.000000	0.704396	
190	Kristoffer Ajer	Norway	DEF	0.011476	0.08	0.000000	0.832243	
65	Jonathan David	Canada	FWD	0.011390	0.02	0.000000	0.744899	

In [128…

```
position_pc_modifier = {
    "GK": 0.08,
    "FWD": 0.01,
    "MID": 0.05,
    "DEF": 0.14,
}

df["position_pc_modifier"] = df["position"].map(position_pc_modifier)

raw_pc = (
    0.20 * df["position_pc_modifier"]
  + 0.15 * df["penalty_conceded_rate"]
  + 0.15 * df["tackles_per90"]
  + 0.35 * df["prob_yellow_card"]
  + 0.15 * df["opp_over_05_prob"]
)

df["prob_penalty_committed"] = 0.08 * sigmoid((raw_pc - 0.5) * 5) * df["prob_plays_60min"]
```

In [128…

```
cols = [
    "name",
    "team",
    "position",
    "prob_penalty_committed",
    "position_pc_modifier",
    "penalty_conceded_rate",
    "tackles_per90",
    "prob_yellow_card",
    "opp_over_05_prob",
    "prob_plays_any",
]

df[cols].sort_values("prob_penalty_committed", ascending=False).head(30)
```

Out[128...

	name	team	position	prob_penalty_committed	position_pc_modifier	penalty_conceded_rate	t
209	Matías Galarza	Paraguay	MID	0.018474	0.05	0.0	
216	Juan José Cáceres	Paraguay	DEF	0.017669	0.14	0.0	
211	Gustavo Gómez	Paraguay	DEF	0.017461	0.14	0.0	
111	Yasser Ibrahim	Egypt	DEF	0.017455	0.14	0.0	
360	Andrés Cubas	Paraguay	MID	0.017425	0.05	0.0	
107	Mohamed Hany	Egypt	DEF	0.016316	0.14	0.0	
227	Orlando Gill	Paraguay	GK	0.015558	0.08	0.0	
212	Júnior Alonso	Paraguay	DEF	0.015271	0.14	0.0	
100	Marwan Attia	Egypt	MID	0.015142	0.05	0.0	
236	Renato Veiga	Portugal	DEF	0.014458	0.14	0.0	
108	Ramy Rabia	Egypt	DEF	0.014303	0.14	0.0	
359	Mohamed Salah	Egypt	MID	0.014276	0.05	0.0	
117	Mostafa Shobeir	Egypt	GK	0.014193	0.08	0.0	
28	Brandon Mechele	Belgium	DEF	0.014149	0.14	0.0	
347	Rúben Dias	Portugal	DEF	0.013972	0.14	0.0	
379	Alistair Johnston	Canada	DEF	0.013937	0.14	0.0	
229	Julio Enciso	Paraguay	MID	0.013863	0.05	0.0	
103	Emam Ashour	Egypt	MID	0.013810	0.05	0.0	
232	Nuno Mendes	Portugal	DEF	0.013800	0.14	0.0	
22	Timothy Castagne	Belgium	DEF	0.013374	0.14	0.0	
112	Omar Marmoush	Egypt	FWD	0.013080	0.01	0.0	
61	Richie Laryea	Canada	DEF	0.013042	0.14	0.0	
279	Nico Elvedi	Switzerland	DEF	0.012773	0.14	0.0	
21	Maxim De Cuyper	Belgium	DEF	0.012660	0.14	0.0	
39	Youri Tielemans	Belgium	MID	0.012597	0.05	0.0	
250	Bruno Fernandes	Portugal	MID	0.012482	0.05	0.0	
277	Manuel Akanji	Switzerland	DEF	0.012357	0.14	0.0	

	name	team	position	prob_penalty_committed	position_pc_modifier	penalty_conceded_rate	t
346	Diogo Costa	Portugal	GK	0.012350	0.08	0.0	
62	Derek Cornelius	Canada	DEF	0.012330	0.14	0.0	
189	Patrick Berg	Norway	MID	0.012275	0.05	0.0	

In [129…

```
df["lambda_opp"] = -np.log(df["team_cs_prob"].clip(lower=0.01))

df["prob_gc_0"] = np.exp(-df["lambda_opp"])
df["prob_gc_1"] = df["lambda_opp"] * np.exp(-df["lambda_opp"])
df["prob_gc_2"] = (df["lambda_opp"] ** 2 / 2) * np.exp(-df["lambda_opp"])
df["prob_gc_3"] = (df["lambda_opp"] ** 3 / 6) * np.exp(-df["lambda_opp"])
df["prob_gc_4plus"] = 1 - (
    df["prob_gc_0"]
    + df["prob_gc_1"]
    + df["prob_gc_2"]
    + df["prob_gc_3"]
)

# RAW expected goals conceded (true intensity)
df["expected_goals_conceded"] = df["lambda_opp"]

# Normalize ONLY as auxiliary feature (separate column)
scaler = MinMaxScaler()
df["expected_goals_conceded_norm"] = scaler.fit_transform(
    df[["expected_goals_conceded"]]
)

# Player exposure to goals conceded (correct)
df["expected_player_goals_conceded"] = (
    df["lambda_opp"] * df["prob_plays_60min"]
)

# GC penalty MUST use raw λ (correct Poisson expectation)
df["expected_gc_penalty"] = (
    -(df["lambda_opp"] - 1 + np.exp(-df["lambda_opp"]))
    * df["prob_plays_60min"]
)

df.loc[df["position"].isin(["MID", "FWD"]), [
    "expected_player_goals_conceded",
    "expected_gc_penalty"
]] = 0.0
```

In [129…

```
cols = [
    "name",
    "team",
    "position",
    "prob_plays_60min",
    "team_cs_prob",
    "lambda_opp",
    "prob_gc_0",
    "prob_gc_1",
    "prob_gc_2",
    "prob_gc_3",
    "prob_gc_4plus",
    "expected_player_goals_conceded",
    "expected_gc_penalty",
]

df[df["position"].isin(["GK", "DEF"])][cols] \
    .sort_values("expected_gc_penalty") \
    .head(30)
```

Out[129...

	name	team	position	prob_plays_60min	team_cs_prob	lambda_opp	prob_gc_0	prob_gc_1	pr
211	Gustavo Gómez	Paraguay	DEF	0.939509	0.1448	1.932402	0.1448	0.279812	0
227	Orlando Gill	Paraguay	GK	0.924142	0.1448	1.932402	0.1448	0.279812	0
216	Juan José Cáceres	Paraguay	DEF	0.886742	0.1448	1.932402	0.1448	0.279812	0
212	Júnior Alonso	Paraguay	DEF	0.802946	0.1448	1.932402	0.1448	0.279812	0
107	Mohamed Hany	Egypt	DEF	0.936805	0.1910	1.655482	0.1910	0.316197	0
111	Yasser Ibrahim	Egypt	DEF	0.923769	0.1910	1.655482	0.1910	0.316197	0
117	Mostafa Shobeir	Egypt	GK	0.922371	0.1910	1.655482	0.1910	0.316197	0
108	Ramy Rabia	Egypt	DEF	0.858397	0.1910	1.655482	0.1910	0.316197	0
232	Nuno Mendes	Portugal	DEF	0.887945	0.2285	1.476219	0.2285	0.337316	0
236	Renato Veiga	Portugal	DEF	0.887781	0.2285	1.476219	0.2285	0.337316	0
346	Diogo Costa	Portugal	GK	0.880352	0.2285	1.476219	0.2285	0.337316	0
347	Rúben Dias	Portugal	DEF	0.847881	0.2285	1.476219	0.2285	0.337316	0
190	Kristoffer Ajer	Norway	DEF	0.782690	0.2170	1.527858	0.2170	0.331545	0
202	Ørjan Nyland	Norway	GK	0.760983	0.2170	1.527858	0.2170	0.331545	0
379	Alistair Johnston	Canada	DEF	0.899531	0.2542	1.369634	0.2542	0.348161	0
194	David Møller Wolfe	Norway	DEF	0.753155	0.2170	1.527858	0.2170	0.331545	0
217	José Canale	Paraguay	DEF	0.511589	0.1448	1.932402	0.1448	0.279812	0
70	Maxime Crépeau	Canada	GK	0.880352	0.2542	1.369634	0.2542	0.348161	0
61	Richie Laryea	Canada	DEF	0.875718	0.2542	1.369634	0.2542	0.348161	0
196	Torbjørn Heggem	Norway	DEF	0.726889	0.2170	1.527858	0.2170	0.331545	0
28	Brandon Mechele	Belgium	DEF	0.934625	0.2731	1.297917	0.2731	0.354461	0
340	Thibaut Courtois	Belgium	GK	0.924142	0.2731	1.297917	0.2731	0.354461	0
192	Marcus Pedersen	Norway	DEF	0.691522	0.2170	1.527858	0.2170	0.331545	0
213	Omar Alderete	Paraguay	DEF	0.472550	0.1448	1.932402	0.1448	0.279812	0
62	Derek Cornelius	Canada	DEF	0.806031	0.2542	1.369634	0.2542	0.348161	0
22	Timothy Castagne	Belgium	DEF	0.868378	0.2731	1.297917	0.2731	0.354461	0
21	Maxim De Cuyper	Belgium	DEF	0.845297	0.2731	1.297917	0.2731	0.354461	0

	name	team	position	prob_plays_60min	team_cs_prob	lambda_opp	prob_gc_0	prob_gc_1	pr
306	Alexander Freeman	USA	DEF	0.813929	0.2686	1.314532	0.2686	0.353083	0
277	Manuel Akanji	Switzerland	DEF	0.898167	0.2956	1.218748	0.2956	0.360262	0
279	Nico Elvedi	Switzerland	DEF	0.890369	0.2956	1.218748	0.2956	0.360262	0

In [129…

```
raw_save = (
    0.35 * df["expected_goals_conceded_norm"]  # opportunity
  + 0.40 * df["saves_per90"]                   # ability
  + 0.05 * df["recent_saves_per90"]            # recent form
  + 0.20 * df["price"]                         # overall GK quality
)

# Expected saves (λ)
df["expected_saves"] = (
    sigmoid((raw_save - 0.5) * 6) * 4
) * df["prob_plays_60min"]

df.loc[df["position"] != "GK", "expected_saves"] = 0.0
```

In [129…

```
cols = [
    "name",
    "team",
    "expected_saves",
    "expected_goals_conceded",
    "saves_per90",
    "recent_saves_per90",
    "price",
    "expected_goals_conceded_norm",
]

df[df["position"] == "GK"][cols] \
    .sort_values("expected_saves", ascending=False) \
    .head(30)
```

Out[129...

	name	team	expected_saves	expected_goals_conceded	saves_per90	recent_saves_per90	
227	Orlando Gill	Paraguay	3.018138	1.932402	0.876923	0.960000	0.0
346	Diogo Costa	Portugal	2.866347	1.476219	0.700000	0.866667	0.9
288	Gregor Kobel	Switzerland	2.391703	1.218748	0.650000	0.666667	0.8
340	Thibaut Courtois	Belgium	2.205682	1.297917	0.415385	0.420000	0.9
56	Alisson Becker	Brazil	2.169107	1.046969	0.550000	0.600000	1.0
117	Mostafa Shobeir	Egypt	1.769626	1.655482	0.461538	0.420000	0.0
202	Ørjan Nyland	Norway	1.668747	1.527858	0.400000	0.600000	0.4
265	Unai Simón	Spain	1.339842	1.046969	0.200000	0.200000	1.0
133	Jordan Pickford	England	1.330272	0.985909	0.300000	0.200000	0.8
311	Matt Freese	USA	1.283220	1.314532	0.333333	0.500000	0.4
70	Maxime Crépeau	Canada	1.181180	1.369634	0.250000	0.200000	0.3
152	Mike Maignan	France	1.152884	0.531709	0.400000	0.400000	1.0
393	Yassine Bounou	Morocco	1.074582	0.830113	0.276923	0.120000	0.8
176	Raúl Rangel	Mexico	0.950666	1.115962	0.310345	0.279070	0.2
10	Emiliano Martínez	Argentina	0.890752	0.606052	0.184615	0.240000	1.0
203	Egil Selvik	Norway	0.817358	1.527858	1.000000	1.000000	0.2
90	Camilo Vargas	Colombia	0.816774	0.947008	0.200000	0.200000	0.5
310	Matt Turner	USA	0.196047	1.314532	0.000000	0.000000	0.3
225	Roberto Fernández	Paraguay	0.186455	1.932402	0.000000	0.000000	0.3
244	Rui Silva	Portugal	0.172304	1.476219	0.000000	0.000000	0.8
55	Ederson	Brazil	0.168434	1.046969	0.000000	0.000000	1.0
245	José Sá	Portugal	0.156367	1.476219	0.000000	0.000000	0.8
226	Gastón Olveira	Paraguay	0.152715	1.932402	0.000000	0.000000	0.2
36	Senne Lammens	Belgium	0.135631	1.297917	0.000000	0.000000	0.1
264	David Raya	Spain	0.130169	1.046969	0.000000	0.000000	1.0
118	Mohamed Alaa	Egypt	0.126392	1.655482	0.000000	0.000000	0.0
115	Mohamed El Shenawy	Egypt	0.111501	1.655482	0.000000	0.000000	0.0
174	Guillermo Ochoa	Mexico	0.107084	1.115962	0.000000	0.000000	0.4

	name	team	expected_saves	expected_goals_conceded	saves_per90	recent_saves_per90	
116	El Mahdy Soliman	Egypt	0.096355	1.655482	0.000000	0.000000	0.(
289	Yvon Mvogo	Switzerland	0.087953	1.218748	0.000000	0.000000	0.4

In [129…
```
raw_pen_save = (
    0.55 * df["price"]
  + 0.30 * df["expected_goals_conceded_norm"]
  + 0.15 * df["stat_PS"]
)

# Strong compression because penalty saves are exceptionally rare
df["prob_penalty_save"] = sigmoid((raw_pen_save - 0.5) * 3) / 20 * df["prob_plays_60min"]

df.loc[df["position"] != "GK", "prob_penalty_save"] = 0.0
```

In [129…
```
cols = [
    "name",
    "team",
    "prob_penalty_save",
    "price",
    "expected_goals_conceded",
    "opp_over_05_prob",
    "prob_plays_60min",
    "stat_PS",
]

df[df["position"] == "GK"][cols] \
    .sort_values("prob_penalty_save", ascending=False) \
    .head(30)
```

Out[129...

	name	team	prob_penalty_save	price	expected_goals_conceded	opp_over_05_prob	prob
340	Thibaut Courtois	Belgium	0.029112	0.933333	1.297917	0.704396	
346	Diogo Costa	Portugal	0.028889	0.933333	1.476219	0.802133	
152	Mike Maignan	France	0.028420	1.000000	0.531709	0.000000	
265	Unai Simón	Spain	0.027203	1.000000	1.046969	0.528108	
56	Alisson Becker	Brazil	0.027203	1.000000	1.046969	0.528108	
10	Emiliano Martínez	Argentina	0.025890	1.000000	0.606052	0.093859	
288	Gregor Kobel	Switzerland	0.024870	0.800000	1.218748	0.653346	
133	Jordan Pickford	England	0.024440	0.866667	0.985909	0.477507	
393	Yassine Bounou	Morocco	0.023239	0.800000	0.830113	0.336836	
117	Mostafa Shobeir	Egypt	0.019312	0.000000	1.655482	0.895719	
202	Ørjan Nyland	Norway	0.018170	0.466667	1.527858	0.832243	
90	Camilo Vargas	Colombia	0.018162	0.533333	0.947008	0.448799	
70	Maxime Crépeau	Canada	0.017542	0.333333	1.369634	0.744899	
311	Matt Freese	USA	0.016874	0.466667	1.314532	0.717678	
227	Orlando Gill	Paraguay	0.016373	0.000000	1.932402	1.000000	
176	Raúl Rangel	Mexico	0.014551	0.266667	1.115962	0.582486	
55	Ederson	Brazil	0.004937	1.000000	1.046969	0.528108	
310	Matt Turner	USA	0.004936	0.333333	1.314532	0.717678	
203	Egil Selvik	Norway	0.004689	0.200000	1.527858	0.832243	
264	David Raya	Spain	0.003815	1.000000	1.046969	0.528108	
244	Rui Silva	Portugal	0.003736	0.800000	1.476219	0.802133	
36	Senne Lammens	Belgium	0.003398	0.733333	1.297917	0.704396	
245	José Sá	Portugal	0.003390	0.800000	1.476219	0.802133	
174	Guillermo Ochoa	Mexico	0.003189	0.466667	1.115962	0.582486	
225	Roberto Fernández	Paraguay	0.003010	0.333333	1.932402	1.000000	
319	Weverton	Brazil	0.002705	0.666667	1.046969	0.528108	
11	Gerónimo Rulli	Argentina	0.002549	0.666667	0.606052	0.093859	
226	Gastón Olveira	Paraguay	0.002425	0.200000	1.932402	1.000000	
289	Yvon Mvogo	Switzerland	0.002400	0.466667	1.218748	0.653346	

	name	team	prob_penalty_save	price	expected_goals_conceded	opp_over_05_prob	prob
118	Mohamed Alaa	Egypt	0.002350	0.000000	1.655482	0.895719	

In [129…
```
raw_tackles = (
    0.50 * df["tackles_per90"]
  + 0.15 * df["recent_tackles_per90"]
  + 0.15 * df["expected_goals_conceded_norm"]
  + 0.20 * df["match_over_25_prob"]
)

df["lambda_tackles"] = -np.log(
    (1 - raw_tackles).clip(lower=0.001)
)

df["expected_tackles"] = (
    df["lambda_tackles"].clip(lower=0)
    * df["prob_plays_60min"]
)

df.loc[df["position"] != "MID", "expected_tackles"] = 0.0
```

In [129…
```
cols = [
    "name",
    "team",
    "expected_tackle_points",
    "tackles_per90",
    "recent_tackles_per90",
    "expected_goals_conceded",
    "match_over_25_prob",
    "price",
    "prob_plays_60min",
]

df[df["position"] == "MID"][cols] \
    .sort_values("expected_tackle_points", ascending=False) \
    .head(30)
```

Out[129...

	name	team	expected_tackle_points	tackles_per90	recent_tackles_per90	expected_goals_co
352	Christian Fassnacht	Switzerland	0.200000	0.200000	1.000000	1.
136	Jordan Henderson	England	0.166667	0.166667	0.833333	0.
158	Maghnes Akliouche	France	0.142857	0.142857	0.714286	0.
337	Luis Chávez	Mexico	0.083333	0.083333	0.000000	1
411	Álex Zendejas	USA	0.076923	0.076923	0.384615	1.
78	Kevin Castaño	Colombia	0.066667	0.066667	0.000000	0.
16	Valentín Barco	Argentina	0.052632	0.052632	0.263158	0.
96	Jaminton Campaz	Colombia	0.052632	0.052632	0.000000	0.
296	Brenden Aaronson	USA	0.051948	0.051948	0.259740	1.
159	Aurélien Tchouaméni	France	0.051852	0.051852	0.305556	0.
58	Danilo	Brazil	0.050000	0.050000	0.500000	1.
231	Samú Costa	Portugal	0.050000	0.050000	0.250000	1.
119	Mohanad Lasheen	Egypt	0.048148	0.048148	0.138889	1.
209	Matías Galarza	Paraguay	0.046980	0.046980	0.234899	1.
17	Exequiel Palacios	Argentina	0.044444	0.044444	0.222222	0.
15	Nico Paz	Argentina	0.042254	0.042254	0.081967	0.
59	Casemiro	Brazil	0.041522	0.041522	0.225410	1.
79	Richard Ríos	Colombia	0.040816	0.040816	0.227273	0.
390	Gavi	Spain	0.039474	0.039474	0.000000	1.
207	Maurício Magalhães Prado	Paraguay	0.039216	0.039216	0.185185	1.
268	Rodri	Spain	0.039216	0.039216	0.222222	1.
401	Neil El Aynaoui	Morocco	0.039164	0.039164	0.187713	0.
368	Fredrik Aursnes	Norway	0.037209	0.037209	0.211268	1.
228	Damián Bobadilla	Paraguay	0.036842	0.036842	0.241379	1.
19	Alexis Mac Allister	Argentina	0.036474	0.036474	0.230126	0.
360	Andrés Cubas	Paraguay	0.035897	0.035897	0.150000	1.
102	Hamdi Fathy	Egypt	0.035714	0.035714	0.138889	1.
276	Fabian Rieder	Switzerland	0.034783	0.034783	0.222222	1.
230	Diego Gómez	Paraguay	0.033898	0.033898	0.192308	1.

	name	team	expected_tackle_points	tackles_per90	recent_tackles_per90	expected_goals_co
292	Denis Zakaria	Switzerland	0.033898	0.033898	0.172414	1.

In [129…

```
raw_cc = (
    0.25 * df["recent_cc_per90"]
  + 0.25 * df["stat_CC"]
  + 0.25 * df["match_over_25_prob"]
  + 0.15 * df["anytime_scorer_prob"]
  + 0.10 * df["price"]
)

df["lambda_cc"] = -np.log(
    (1 - raw_cc).clip(lower=0.001)
)

df["expected_chances_created"] = (
    df["lambda_cc"].clip(lower=0)
    * df["prob_plays_60min"]
)

df.loc[df["position"] != "MID", "expected_chances_created"] = 0.0
```

In [129…

```
cols = [
    "name",
    "team",
    "position",
    "expected_cc_points",
    "cc_per90",
    "recent_cc_per90",
    "stat_CC",
    "match_over_25_prob",
    "anytime_scorer_prob",
    "price",
    "prob_plays_60min",
]

df[cols].sort_values("expected_cc_points", ascending=False).head(30)
```

Out[129...

	name	team	position	expected_cc_points	cc_per90	recent_cc_per90	stat_CC	match_over_
327	Cucho Hernández	Colombia	FWD	1.000000	1.000000	0.000000	0.2	
95	Juan Fernando Quintero	Colombia	MID	0.317460	0.317460	0.412698	0.4	
74	Niko Sigur	Canada	MID	0.263158	0.263158	0.342105	0.2	
160	Rayan Cherki	France	MID	0.181818	0.181818	0.250000	0.2	
405	Soufiane Rahimi	Morocco	FWD	0.163934	0.163934	0.216667	0.2	
161	Michael Olise	France	MID	0.162338	0.162338	0.178899	1.0	
406	Chemsdine Talbi	Morocco	FWD	0.156250	0.156250	0.333333	0.2	
271	Johan Manzambi	Switzerland	MID	0.150000	0.150000	0.148571	0.6	
50	Gabriel Martinelli	Brazil	MID	0.135135	0.135135	0.175676	0.2	
295	Sebastian Berhalter	USA	MID	0.129870	0.129870	0.238532	0.4	
189	Patrick Berg	Norway	MID	0.128205	0.128205	0.173333	0.6	
60	Lucas Paquetá	Brazil	MID	0.127119	0.127119	0.222857	0.6	
5	Lionel Messi	Argentina	FWD	0.125000	0.125000	0.216667	0.8	
37	Nicolas Raskin	Belgium	MID	0.124224	0.124224	0.102362	0.4	
132	Noni Madueke	England	FWD	0.123457	0.123457	0.152047	0.6	
355	Karim Hafez	Egypt	DEF	0.121951	0.121951	0.162500	0.2	
313	Christian Pulisic	USA	MID	0.121212	0.121212	0.000000	0.4	
139	Declan Rice	England	MID	0.119522	0.119522	0.145251	0.6	
171	Roberto Alvarado	Mexico	FWD	0.117647	0.117647	0.156000	0.8	
85	Santiago Arias	Colombia	DEF	0.114943	0.114943	0.149425	0.2	
367	Thelo Aasgaard	Norway	MID	0.111111	0.111111	0.144444	0.2	
75	Nathan Saliba	Canada	MID	0.109890	0.109890	0.142857	0.4	
357	Mostafa Zico	Egypt	MID	0.101695	0.101695	0.178082	0.6	
382	Stephen Eustaquio	Canada	MID	0.099668	0.099668	0.122642	0.6	
191	Julian Ryerson	Norway	DEF	0.097087	0.097087	1.000000	0.2	
338	Luis Romo	Mexico	MID	0.096618	0.096618	0.125604	0.4	
140	Jude Bellingham	England	MID	0.095541	0.095541	0.166667	0.6	
53	Rayan	Brazil	FWD	0.090090	0.090090	0.117117	0.4	

	name	team	position	expected_cc_points	cc_per90	recent_cc_per90	stat_CC	match_over_2
321	Bruno Guimarães	Brazil	MID	0.088235	0.088235	0.150000	0.6	
232	Nuno Mendes	Portugal	DEF	0.087977	0.087977	0.144981	0.6	

In [130…

```
raw_sot = (
    0.35 * df["sot_per90"]
  + 0.15 * df["stat_GS"]
  + 0.20 * df["match_over_25_prob"]
  + 0.20 * df["anytime_scorer_prob"]
  + 0.10 * df["price"]
)

df["lambda_sot"] = -np.log(
    (1 - raw_sot).clip(lower=0.001)
)

df["expected_shots_on_target"] = (
    df["lambda_sot"].clip(lower=0)
    * df["prob_plays_60min"]
)

df.loc[df["position"] != "FWD", "expected_shots_on_target"] = 0.0
```

In [130…

```
cols = [
    "name",
    "team",
    "position",
    "expected_sot_points",
    "sot_per90",
    "goal_rate",
    "stat_GS",
    "match_over_25_prob",
    "anytime_scorer_prob",
    "price",
    "prob_plays_60min",
]

df[cols].sort_values("expected_sot_points", ascending=False).head(30)
```

Out[130…

	name	team	position	expected_sot_points	sot_per90	goal_rate	stat_GS	match_over_25_
96	Jaminton Campaz	Colombia	MID	1.000000	1.000000	1.000000	0.142857	0.
323	Neymar	Brazil	MID	0.678571	0.678571	0.000000	0.000000	0.
365	Ayoube Amaimouni-Echghouyab	Morocco	FWD	0.593750	0.593750	0.000000	0.000000	0.
58	Danilo	Brazil	MID	0.475000	0.475000	0.000000	0.000000	0.
385	Promise David	Canada	FWD	0.445312	0.445312	0.296875	0.142857	0.
5	Lionel Messi	Argentina	FWD	0.445312	0.445312	0.415625	1.000000	0.
148	Kylian Mbappé	France	FWD	0.351852	0.351852	0.324786	0.857143	0.
198	Erling Haaland	Norway	FWD	0.316667	0.316667	0.351852	0.714286	0.
405	Soufiane Rahimi	Morocco	FWD	0.311475	0.311475	0.311475	0.142857	0.
238	Gonçalo Ramos	Portugal	FWD	0.279412	0.279412	0.558824	0.142857	0
183	Ayoub El Kaabi	Morocco	FWD	0.271429	0.271429	0.000000	0.000000	0.
48	Vinícius Júnior	Brazil	MID	0.270655	0.270655	0.216524	0.571429	0.
259	Lamine Yamal	Spain	MID	0.253333	0.253333	0.084444	0.142857	0
261	Mikel Oyarzabal	Spain	FWD	0.252492	0.252492	0.252492	0.571429	0
127	Harry Kane	England	FWD	0.241525	0.241525	0.268362	0.714286	0.
66	Cyle Larin	Canada	FWD	0.234568	0.234568	0.234568	0.285714	0.
151	Désiré Doué	France	MID	0.221963	0.221963	0.088785	0.142857	0.
104	Mahmoud Saber	Egypt	MID	0.206522	0.206522	0.413043	0.142857	0.
287	Dan Ndoye	Switzerland	FWD	0.203863	0.203863	0.081545	0.142857	0.
49	Matheus Cunha	Brazil	FWD	0.202128	0.202128	0.242553	0.428571	0.
65	Jonathan David	Canada	FWD	0.200906	0.200906	0.172205	0.428571	0.
113	Trezeguet	Egypt	FWD	0.197917	0.197917	0.131944	0.142857	0.
21	Maxim De Cuyper	Belgium	DEF	0.195205	0.195205	0.000000	0.000000	0.
305	Auston Trusty	USA	DEF	0.190000	0.190000	0.190000	0.142857	0.
350	Cristiano Ronaldo	Portugal	FWD	0.189459	0.189459	0.162393	0.428571	0
207	Maurício Magalhães Prado	Paraguay	MID	0.186275	0.186275	0.124183	0.142857	0.
101	Zizo	Egypt	MID	0.182692	0.182692	0.000000	0.000000	0.
140	Jude Bellingham	England	MID	0.181529	0.181529	0.121019	0.285714	0.
339	Santiago Giménez	Mexico	FWD	0.179245	0.179245	0.000000	0.000000	0.
168	Raúl Jiménez	Mexico	FWD	0.165217	0.165217	0.165217	0.285714	0.

In [130…
```
df["expected_qualification_points"] = df["team_qualify_prob"] * 2
```

In [130…
```
cols = [
    "name",
    "team",
    "position",
    "expected_qualification_points",
    "team_qualify_prob",
    "price",
    "prob_plays_60min",
]

df[cols].sort_values("expected_qualification_points", ascending=False).head(30)
```

Out[130…

	name	team	position	expected_qualification_points	team_qualify_prob	price	prob_plays_
332	Jean-Philippe Mateta	France	FWD	1.886792	0.943396	0.384615	0.1
329	William Saliba	France	DEF	1.886792	0.943396	0.720000	0.7
330	Jules Koundé	France	DEF	1.886792	0.943396	0.760000	0.8
331	Bradley Barcola	France	MID	1.886792	0.943396	0.655172	0.7
153	Brice Samba	France	GK	1.886792	0.943396	0.666667	0.1
157	Manu Koné	France	MID	1.886792	0.943396	0.327586	0.4
155	Adrien Rabiot	France	MID	1.886792	0.943396	0.379310	0.7
141	Theo Hernández	France	DEF	1.886792	0.943396	0.600000	0.4
328	Robin Risser	France	GK	1.886792	0.943396	0.000000	0.1
152	Mike Maignan	France	GK	1.886792	0.943396	1.000000	0.8
151	Désiré Doué	France	MID	1.886792	0.943396	0.568966	0.4
150	Marcus Thuram	France	FWD	1.886792	0.943396	0.538462	0.1
149	Ousmane Dembélé	France	MID	1.886792	0.943396	1.000000	0.8
148	Kylian Mbappé	France	FWD	1.886792	0.943396	1.000000	0.8
147	Malo Gusto	France	DEF	1.886792	0.943396	0.640000	0.1
146	Lucas Digne	France	DEF	1.886792	0.943396	0.600000	0.6
145	Lucas Hernández	France	DEF	1.886792	0.943396	0.680000	0.1
144	Ibrahima Konaté	France	DEF	1.886792	0.943396	0.560000	0.1
156	Warren Zaïre-Emery	France	MID	1.886792	0.943396	0.327586	0.1
142	Maxence Lacroix	France	DEF	1.886792	0.943396	0.400000	0.2
143	Dayot Upamecano	France	DEF	1.886792	0.943396	0.720000	0.8
160	Rayan Cherki	France	MID	1.886792	0.943396	0.655172	0.2
159	Aurélien Tchouaméni	France	MID	1.886792	0.943396	0.396552	0.6
158	Maghnes Akliouche	France	MID	1.886792	0.943396	0.379310	0.1
161	Michael Olise	France	MID	1.886792	0.943396	0.913793	0.8
154	N'Golo Kanté	France	MID	1.886792	0.943396	0.155172	0.1
9	José Manuel López	Argentina	FWD	1.754386	0.877193	0.200000	0.1

	name	team	position	expected_qualification_points	team_qualify_prob	price	prob_plays_6
12	Juan Musso	Argentina	GK	1.754386	0.877193	0.533333	0.1
11	Gerónimo Rulli	Argentina	GK	1.754386	0.877193	0.666667	0.1
6	Julián Alvarez	Argentina	FWD	1.754386	0.877193	0.707692	0.5

In [130…
```
df.to_csv(OUT_CSV, index=False)

print(f"\nSaved {OUT_CSV} {df.shape[0]} rows x {df.shape[1]} cols")
```

```
Saved fantasy_enriched.csv 414 rows x 250 cols
```

In [130…
```
BUDGET = 105.0
MAX_PER_COUNTRY = 4
SQUAD_SIZE = 15
XI_SIZE = 11
BENCH_WEIGHT = 0.6

def compute_total_expected_points(df):
    df = df.copy()

    cs_value = {"GK": 5, "DEF": 5, "MID": 1, "FWD": 0}
    goal_value = {"GK": 9, "DEF": 7, "MID": 6, "FWD": 5}

    df["goal_pts_value"] = df["position"].map(goal_value)
    df["cs_pts_value"] = df["position"].map(cs_value)

    # appearance: 1pt for playing any, +1 if 60+
    e_appearance = df["prob_plays_any"] * 1 + df["prob_plays_60min"] * 1

    # goal
    e_goal = df["prob_scores"] * df["goal_pts_value"]

    # assist
    e_assist = df["expected_assists"] * 3

    # clean sheet: gated by 60min, value depends on position
    e_cs = df["prob_clean_sheet"] * df["cs_pts_value"]

    # goals conceded penalty: GK and DEF only
    e_gc = df["expected_gc_penalty"].copy()
    e_gc[~df["position"].isin(["GK", "DEF"])] = 0.0

    # saves bonus: GK only, every 3 saves = +1
    e_saves = (df["expected_saves"] / 3).copy()
    e_saves[df["position"] != "GK"] = 0.0

    # penalty save: GK only
    e_pen_save = (df["prob_penalty_save"] * 3).copy()
    e_pen_save[df["position"] != "GK"] = 0.0

    # tackles bonus: MID only, every 3 tackles = +1
    e_tackles = df["expected_tackle_points"].copy()
    e_tackles[df["position"] != "MID"] = 0.0

    # big chances created: MID only, every 2 = +1
    e_cc = df["expected_cc_points"].copy()
    e_cc[df["position"] != "MID"] = 0.0

    # shots on target: FWD only, every 2 = +1
    e_sot = df["expected_sot_points"].copy()
    e_sot[df["position"] != "FWD"] = 0.0

    # yellow card
    e_yc = df["prob_yellow_card"] * -1

    # red card
    e_rc = df["prob_red_card"] * -2

    # own goal
    e_og = df["prob_own_goal"] * -2
```

```
    # penalty won
    e_pw = df["prob_pen_won"] * 2

    # penalty committed
    e_pc = df["prob_penalty_committed"] * -1

    # qualification bonus: +2 per player in XI who advances, requires playing
    # captain's quali bonus is NOT doubled per rules
    e_quali = df["expected_qualification_points"] * df["prob_plays_any"]

    # scouting bonus: +2 if differential==1 AND player scores >4 base pts
    # model: use Poisson confidence interval on expected base points
    # base pts = everything except scouting and captain multiplier
    base_ep = (
        e_appearance + e_goal + e_assist + e_cs + e_gc +
        e_saves + e_pen_save + e_tackles + e_cc + e_sot +
        e_yc + e_rc + e_og + e_pw + e_pc + e_quali
    )

    # Poisson 90% CI: lower = ppf(0.05, mu), upper = ppf(0.95, mu)
    mu = base_ep.clip(lower=0.01)
    lower_ci = pd.Series(poisson.ppf(0.05, mu), index=df.index)
    upper_ci = pd.Series(poisson.ppf(0.95, mu), index=df.index)

    # if lower CI > 4: full +2 expected
    # if upper CI < 4: 0
    # if CI straddles 4: scale by P(X>4 | mu) * 2
    p_exceeds_4 = pd.Series(1 - poisson.cdf(4, mu), index=df.index)

    scouting_ep = pd.Series(0.0, index=df.index)
    eligible = df["differential"] == 1
    full_bonus = eligible & (lower_ci > 4)
    no_bonus = eligible & (upper_ci < 4)
    partial = eligible & ~full_bonus & ~no_bonus

    scouting_ep[full_bonus] = 2.0
    scouting_ep[no_bonus] = 0.0
    scouting_ep[partial] = p_exceeds_4[partial] * 2.0

    df["e_appearance"] = e_appearance
    df["e_goal"] = e_goal
    df["e_assist"] = e_assist
    df["e_cs"] = e_cs
    df["e_gc"] = e_gc
    df["e_saves"] = e_saves
    df["e_pen_save"] = e_pen_save
    df["e_tackles"] = e_tackles
    df["e_cc"] = e_cc
    df["e_sot"] = e_sot
    df["e_yc"] = e_yc
    df["e_rc"] = e_rc
    df["e_og"] = e_og
    df["e_pw"] = e_pw
    df["e_pc"] = e_pc
    df["e_quali"] = e_quali
    df["e_scouting"] = scouting_ep
    df["expected_points"] = base_ep + scouting_ep

    return df
```

In [130…

```
def optimize_squad(df, budget=BUDGET, max_per_country=MAX_PER_COUNTRY):
    idx = df.index.tolist()

    x = LpVariable.dicts("squad", idx, cat="Binary")
    s = LpVariable.dicts("xi", idx, cat="Binary")
    b = LpVariable.dicts("bench", idx, cat="Binary")
    cap = LpVariable.dicts("cap", idx, cat="Binary")

    model = LpProblem("fantasy", LpMaximize)


    model += lpSum(
        df.loc[i, "expected_points"] * s[i]
        + df.loc[i, "expected_points"] * cap[i]
```

```
        + df.loc[i, "expected_points"] * BENCH_WEIGHT * b[i]
        for i in idx
    )

    # squad = 15, xi = 11, bench = 4
    model += lpSum(x[i] for i in idx) == 15
    model += lpSum(s[i] for i in idx) == 11
    model += lpSum(b[i] for i in idx) == 4

    for i in idx:
        model += s[i] + b[i] == x[i]

    # budget
    model += lpSum(df.loc[i, "price_raw"] * x[i] for i in idx) <= budget

    # squad composition: 2 GK, 5 DEF, 5 MID, 3 FWD
    for pos, count in [("GK", 2), ("DEF", 5), ("MID", 5), ("FWD", 3)]:
        p_idx = df[df["position"] == pos].index.tolist()
        model += lpSum(x[i] for i in p_idx) == count

    # country limit
    for country in df["team"].unique():
        c_idx = df[df["team"] == country].index.tolist()
        model += lpSum(x[i] for i in c_idx) <= max_per_country

    # XI formation: 1 GK starts, valid outfield formation
    gk_idx = df[df["position"] == "GK"].index.tolist()
    model += lpSum(s[i] for i in gk_idx) == 1
    model += lpSum(b[i] for i in gk_idx) == 1

    for pos, lo, hi in [("DEF", 3, 5), ("MID", 3, 5), ("FWD", 1, 3)]:
        p_idx = df[df["position"] == pos].index.tolist()
        model += lpSum(s[i] for i in p_idx) >= lo
        model += lpSum(s[i] for i in p_idx) <= hi

    # captain: exactly 1, must be in XI
    model += lpSum(cap[i] for i in idx) == 1
    for i in idx:
        model += cap[i] <= s[i]

    model.solve(PULP_CBC_CMD(msg=0))

    if LpStatus[model.status] != "Optimal":
        print(f"Solver status: {LpStatus[model.status]}")
        return None

    in_squad = [i for i in idx if value(x[i]) > 0.5]
    in_xi = [i for i in idx if value(s[i]) > 0.5]
    on_bench = [i for i in idx if value(b[i]) > 0.5]
    captain = next(i for i in idx if value(cap[i]) > 0.5)

    return {
        "squad": df.loc[in_squad].copy(),
        "xi": df.loc[in_xi].copy(),
        "bench": df.loc[on_bench].copy(),
        "captain": captain,
        "total_ep": value(model.objective),
        "cost": df.loc[in_squad, "price_raw"].sum(),
    }
```

In [130…

```
def optimize_ideal_xi(df, max_per_country=MAX_PER_COUNTRY):
    # best possible 11 ignoring bench constraint, just 11 players
    idx = df.index.tolist()

    s = LpVariable.dicts("xi", idx, cat="Binary")
    cap = LpVariable.dicts("cap", idx, cat="Binary")

    model = LpProblem("ideal_xi", LpMaximize)

    # optimize_ideal_xi objective — NO bench term
    model += lpSum(
        df.loc[i, "expected_points"] * s[i]
        + df.loc[i, "expected_points"] * cap[i]
        for i in idx
    )
```

```
    model += lpSum(s[i] for i in idx) == 11

    # no budget constraint for ideal XI comparison
    # country limit still applies
    for country in df["team"].unique():
        c_idx = df[df["team"] == country].index.tolist()
        model += lpSum(s[i] for i in c_idx) <= max_per_country

    gk_idx = df[df["position"] == "GK"].index.tolist()
    model += lpSum(s[i] for i in gk_idx) == 1

    for pos, lo, hi in [("DEF", 3, 5), ("MID", 3, 5), ("FWD", 1, 3)]:
        p_idx = df[df["position"] == pos].index.tolist()
        model += lpSum(s[i] for i in p_idx) >= lo
        model += lpSum(s[i] for i in p_idx) <= hi

    model += lpSum(cap[i] for i in idx) == 1
    for i in idx:
        model += cap[i] <= s[i]

    model.solve(PULP_CBC_CMD(msg=0))

    if LpStatus[model.status] != "Optimal":
        return None

    in_xi = [i for i in idx if value(s[i]) > 0.5]
    captain = next(i for i in idx if value(cap[i]) > 0.5)

    return {
        "xi": df.loc[in_xi].copy(),
        "captain": captain,
        "total_ep": value(model.objective),
    }


def print_team(result, ideal=None):
    pos_order = {"GK": 0, "DEF": 1, "MID": 2, "FWD": 3}
    cap = result["captain"]

    xi = result["xi"].copy()
    xi["_ord"] = xi["position"].map(pos_order)
    xi = xi.sort_values(["_ord", "expected_points"], ascending=[True, False])

    bench = result["bench"].copy()
    bench["_ord"] = bench["position"].map(pos_order)
    bench = bench.sort_values(["_ord", "expected_points"], ascending=[True, False])

    print(f"\nExpected points: {result['total_ep']:.2f}   Cost: ${result['cost']:.1f}M\n")

    print("Starting XI")
    for i, row in xi.iterrows():
        tag = " [C]" if i == cap else ""
        scout = " [SCOUT]" if row.get("differential", 0) == 1 else ""
        print(f"  {row['position']:3}  {row['name']:<28} {row['team']:<22} ${row['price_raw']:.1

    print("\nBench")
    for rank, (i, row) in enumerate(bench.iterrows(), 1):
        print(f"  [{rank}] {row['position']:3}  {row['name']:<28} {row['team']:<22} ${row['price

    print(f"\nCountry breakdown: {dict(result['squad']['team'].value_counts())}")

    if ideal:
        ideal_xi = ideal["xi"].copy()
        ideal_xi["_ord"] = ideal_xi["position"].map(pos_order)
        ideal_xi = ideal_xi.sort_values(["_ord", "expected_points"], ascending=[True, False])
        ideal_cap = ideal["captain"]

        print(f"\nIdeal XI (no budget constraint, 11 only, EP: {ideal['total_ep']:.2f})")
        for i, row in ideal_xi.iterrows():
            tag = " [C]" if i == ideal_cap else ""
            in_squad = i in result["squad"].index
            flag = "" if in_squad else " [NOT IN SQUAD]"
            print(f"  {row['position']:3}  {row['name']:<28} {row['team']:<22} ${row['price_raw'
```

```
if __name__ == "__main__":
    df = pd.read_csv("fantasy_enriched.csv")
    df = df[df["status"] == "playing"].copy()
    df = df.reset_index(drop=True)

    df = compute_total_expected_points(df)

    result = optimize_squad(df, budget=BUDGET, max_per_country=MAX_PER_COUNTRY)
    ideal = optimize_ideal_xi(df, max_per_country=MAX_PER_COUNTRY)

    print_team(result, ideal)
```

```
Expected points: 127.73   Cost: $105.0M

Starting XI
  GK   Emiliano Martínez           Argentina             $5.0  EP:8.19
  DEF  Achraf Hakimi               Morocco               $6.0  EP:7.74
  DEF  Lisandro Martínez           Argentina             $4.6  EP:7.17
  DEF  Nahuel Molina               Argentina             $4.4  EP:6.93 [SCOUT]
  MID  Ousmane Dembélé             France                $10.0  EP:9.60
  MID  Michael Olise               France                $9.5  EP:9.37
  MID  Vinícius Júnior             Brazil                $10.0  EP:8.95
  MID  Ismael Saibari              Morocco               $6.8  EP:7.77
  FWD  Kylian Mbappé               France                $10.5  EP:12.77 [C]
  FWD  Lionel Messi                Argentina             $10.0  EP:11.78
  FWD  Mikel Oyarzabal             Spain                 $8.1  EP:8.22

Bench
  [1] GK   Mike Maignan               France                $5.0  EP:7.20
  [2] DEF  Dávinson Sánchez           Colombia              $4.3  EP:6.63
  [3] DEF  Noussair Mazraoui          Morocco               $4.4  EP:6.13
  [4] MID  Brahim Díaz                Morocco               $6.4  EP:7.48

Country breakdown: {'Argentina': np.int64(4), 'France': np.int64(4), 'Morocco': np.int64(4), 'Br
azil': np.int64(1), 'Colombia': np.int64(1), 'Spain': np.int64(1)}

Ideal XI (no budget constraint, 11 only, EP: 111.92)
  GK   Emiliano Martínez           Argentina             $5.0  EP:8.19
  DEF  Achraf Hakimi               Morocco               $6.0  EP:7.74
  DEF  Lisandro Martínez           Argentina             $4.6  EP:7.17
  DEF  Nahuel Molina               Argentina             $4.4  EP:6.93
  MID  Ousmane Dembélé             France                $10.0  EP:9.60
  MID  Michael Olise               France                $9.5  EP:9.37
  MID  Vinícius Júnior             Brazil                $10.0  EP:8.95
  MID  Bradley Barcola             France                $8.0  EP:8.42 [NOT IN SQUAD]
  FWD  Kylian Mbappé               France                $10.5  EP:12.77 [C]
  FWD  Lionel Messi                Argentina             $10.0  EP:11.78
  FWD  Mikel Oyarzabal             Spain                 $8.1  EP:8.22
```

In [130…

```
def optimize_with_transfers(df, current_team_names, free_transfers=4, budget=BUDGET, max_per_cou

    current_idx = set()
    unmatched = []
    for name in current_team_names:
        match = df[df["name"].str.lower().str.strip() == name.lower().strip()]
        if len(match) == 0:
            match = df[df["name"].str.lower().str.contains(name.lower().strip(), na=False)]
        if len(match) > 0:
            current_idx.add(match.index[0])
        else:
            unmatched.append(name)


    if unmatched:
        print(f"Eliminated players: {unmatched}")

    forced_transfers = len(unmatched)

    idx = df.index.tolist()

    x         = LpVariable.dicts("squad", idx, cat="Binary")
    s         = LpVariable.dicts("xi",    idx, cat="Binary")
    b         = LpVariable.dicts("bench", idx, cat="Binary")
    cap       = LpVariable.dicts("cap",   idx, cat="Binary")
    transfer_out = LpVariable.dicts("out", idx, cat="Binary")
```

```
model = LpProblem("fantasy_transfers", LpMaximize)

transfers_made = lpSum(transfer_out[i] for i in idx)   # voluntary drops only

n_extra = LpVariable("n_extra", lowBound=0)

# CHANGE 2: total transfers = voluntary + forced. Penalty fires when
# (voluntary + forced) > free_transfers, not just voluntary > free_transfers.
model += n_extra >= transfers_made + forced_transfers - free_transfers

model += (
    lpSum(
        df.loc[i, "expected_points"] * s[i]
        + df.loc[i, "expected_points"] * cap[i]
        + df.loc[i, "expected_points"] * BENCH_WEIGHT * b[i]
        for i in idx
    )
    - 3 * n_extra
)

for i in idx:
    if i in current_idx:
        model += transfer_out[i] == 1 - x[i]
    else:
        model += transfer_out[i] == 0

model += lpSum(x[i] for i in idx) == 15
model += lpSum(s[i] for i in idx) == 11
model += lpSum(b[i] for i in idx) == 4

for i in idx:
    model += s[i] + b[i] == x[i]

model += lpSum(df.loc[i, "price_raw"] * x[i] for i in idx) <= budget

for pos, count in [("GK", 2), ("DEF", 5), ("MID", 5), ("FWD", 3)]:
    p_idx = df[df["position"] == pos].index.tolist()
    model += lpSum(x[i] for i in p_idx) == count

for country in df["team"].unique():
    c_idx = df[df["team"] == country].index.tolist()
    model += lpSum(x[i] for i in c_idx) <= max_per_country

gk_idx = df[df["position"] == "GK"].index.tolist()
model += lpSum(s[i] for i in gk_idx) == 1
model += lpSum(b[i] for i in gk_idx) == 1

for pos, lo, hi in [("DEF", 3, 5), ("MID", 3, 5), ("FWD", 1, 3)]:
    p_idx = df[df["position"] == pos].index.tolist()
    model += lpSum(s[i] for i in p_idx) >= lo
    model += lpSum(s[i] for i in p_idx) <= hi

model += lpSum(cap[i] for i in idx) == 1
for i in idx:
    model += cap[i] <= s[i]

model.solve(PULP_CBC_CMD(msg=0))

if LpStatus[model.status] != "Optimal":
    print(f"Solver status: {LpStatus[model.status]}")
    return None

in_squad  = [i for i in idx if value(x[i]) > 0.5]
in_xi     = [i for i in idx if value(s[i]) > 0.5]
on_bench  = [i for i in idx if value(b[i]) > 0.5]
captain   = next(i for i in idx if value(cap[i]) > 0.5)

new_squad_idx  = set(in_squad)
voluntary_out  = [i for i in current_idx if i not in new_squad_idx]
transfers_in   = [i for i in new_squad_idx if i not in current_idx]

# CHANGE 3: total = voluntary drops of matched players + forced drops of eliminated.
n_transfers = len(voluntary_out) + forced_transfers
penalty     = max(0, n_transfers - free_transfers) * 3
```

```
    return {
        "squad":         df.loc[in_squad].copy(),
        "xi":            df.loc[in_xi].copy(),
        "bench":         df.loc[on_bench].copy(),
        "captain":       captain,
        "total_ep":      value(model.objective),
        "cost":          df.loc[in_squad, "price_raw"].sum(),
        "transfers_out": df.loc[voluntary_out, "name"].tolist() + unmatched,  # eliminated shown
        "transfers_in":  df.loc[transfers_in, "name"].tolist(),
        "n_transfers":   n_transfers,
        "penalty":       penalty,
        "forced_out":    unmatched,   # separate field for clarity
    }

def print_transfers(result):
    if result is None:
        print("No solution.")
        return

    print(f"\nTransfers made: {result['n_transfers']} ({result['n_transfers'] - 4} penalised at
    print(f"Point penalty: -{result['penalty']}")

    if result["transfers_out"]:
        print("\nOUT:")
        for name in result["transfers_out"]:
            print(f"  {name}")
        print("IN:")
        for name in result["transfers_in"]:
            print(f"  {name}")

    print_team(result)


if __name__ == "__main__":
    df = pd.read_csv("fantasy_enriched.csv")
    df = df[df["status"] == "playing"].copy()
    df = df.reset_index(drop=True)
    df = compute_total_expected_points(df)

    current_team = [
        "Jordan Pickford", "Lisandro Martínez", "Sergiño Dest", "Nico O'Reilly", "Marc Cucurella
        "Ousmane Dembélé", "Jamal Musiala", "Vinícius Júnior", "Christian Pulisic", "Kylian Mbap
        "Camilo Vargas", "Facundo Medina", "Lionel Messi", "Johan Manzambi"
    ]

    result = optimize_with_transfers(df, current_team, free_transfers=4, budget=BUDGET, max_per_
    ideal = optimize_ideal_xi(df, max_per_country=MAX_PER_COUNTRY)
    print_transfers(result)

    print("\nIdeal XI for comparison:")
    if ideal:
        pos_order = {"GK": 0, "DEF": 1, "MID": 2, "FWD": 3}
        xi = ideal["xi"].copy()
        xi["_ord"] = xi["position"].map(pos_order)
        xi = xi.sort_values(["_ord", "expected_points"], ascending=[True, False])
        cap = ideal["captain"]
        for i, row in xi.iterrows():
            tag = " [C]" if i == cap else ""
            in_squad = i in result["squad"].index if result else False
            flag = "" if in_squad else " [NOT IN SQUAD]"
            print(f"  {row['position']:3}  {row['name']:<28} {row['team']:<22} ${row['price_raw'
```

```
Eliminated players: ['Jamal Musiala']

Transfers made: 4 (all free)
Point penalty: -0

OUT:
  Jordan Pickford
  Sergiño Dest
  Harry Kane
  Jamal Musiala
IN:
  Michael Olise
  Mikel Oyarzabal
  Emiliano Martínez
  Achraf Hakimi

Expected points: 122.07   Cost: $104.4M

Starting XI
  GK   Emiliano Martínez           Argentina             $5.0  EP:8.19
  DEF  Achraf Hakimi               Morocco               $6.0  EP:7.74
  DEF  Lisandro Martínez           Argentina             $4.6  EP:7.17
  DEF  Marc Cucurella              Spain                 $5.1  EP:6.35
  MID  Ousmane Dembélé             France                $10.0  EP:9.60
  MID  Michael Olise               France                $9.5  EP:9.37
  MID  Vinícius Júnior             Brazil                $10.0  EP:8.95
  MID  Christian Pulisic           USA                   $7.0  EP:6.28 [SCOUT]
  FWD  Kylian Mbappé               France                $10.5  EP:12.77 [C]
  FWD  Lionel Messi                Argentina             $10.0  EP:11.78
  FWD  Mikel Oyarzabal             Spain                 $8.1  EP:8.22

Bench
  [1] GK   Camilo Vargas               Colombia              $4.3  EP:6.00
  [2] DEF  Facundo Medina              Argentina             $4.0  EP:5.90
  [3] DEF  Nico O'Reilly               England               $4.7  EP:4.72
  [4] MID  Johan Manzambi              Switzerland           $5.6  EP:4.83

Country breakdown: {'Argentina': np.int64(4), 'France': np.int64(3), 'Spain': np.int64(2), 'Braz
il': np.int64(1), 'Colombia': np.int64(1), 'England': np.int64(1), 'Switzerland': np.int64(1), '
USA': np.int64(1), 'Morocco': np.int64(1)}

Ideal XI for comparison:
  GK   Emiliano Martínez           Argentina             $5.0  EP:8.19
  DEF  Achraf Hakimi               Morocco               $6.0  EP:7.74
  DEF  Lisandro Martínez           Argentina             $4.6  EP:7.17
  DEF  Nahuel Molina               Argentina             $4.4  EP:6.93 [NOT IN SQUAD]
  MID  Ousmane Dembélé             France                $10.0  EP:9.60
  MID  Michael Olise               France                $9.5  EP:9.37
  MID  Vinícius Júnior             Brazil                $10.0  EP:8.95
  MID  Bradley Barcola             France                $8.0  EP:8.42 [NOT IN SQUAD]
  FWD  Kylian Mbappé               France                $10.5  EP:12.77 [C]
  FWD  Lionel Messi                Argentina             $10.0  EP:11.78
  FWD  Mikel Oyarzabal             Spain                 $8.1  EP:8.22
```

In [130…

```python
for pos in ["GK", "DEF", "MID", "FWD"]:
    print(f"\n--- {pos} ---")
    print(df[df["position"] == pos].nlargest(10, "expected_points")[["name", "team", "price_raw"
```

```
--- GK ---
             name        team  price_raw  expected_points
Emiliano Martínez   Argentina        5.0         8.192177
     Mike Maignan      France        5.0         7.195697
    Camilo Vargas    Colombia        4.3         5.995706
   Yassine Bounou     Morocco        4.7         5.552105
   Alisson Becker      Brazil        5.0         5.322927
     Gregor Kobel Switzerland        4.7         5.203078
       Unai Simón       Spain        5.0         5.180738
  Jordan Pickford     England        4.8         4.876853
 Thibaut Courtois     Belgium        4.9         4.491633
      Matt Freese         USA        4.2         4.082214

--- DEF ---
             name      team  price_raw  expected_points
    Achraf Hakimi   Morocco        6.0         7.741311
  Dayot Upamecano    France        5.3         7.483832
Lisandro Martínez Argentina        4.6         7.173978
     Jules Koundé    France        5.4         7.157949
    Nahuel Molina Argentina        4.4         6.925150
 Dávinson Sánchez  Colombia        4.3         6.632040
  Cristian Romero Argentina        4.9         6.607669
   Marc Cucurella     Spain        5.1         6.349981
   William Saliba    France        5.3         6.342404
Noussair Mazraoui   Morocco        4.4         6.126241

--- MID ---
           name      team  price_raw  expected_points
Ousmane Dembélé    France       10.0         9.598307
  Michael Olise    France        9.5         9.372428
Vinícius Júnior    Brazil       10.0         8.949577
Bradley Barcola    France        8.0         8.423811
 Ismael Saibari   Morocco        6.8         7.765328
    Brahim Díaz   Morocco        6.4         7.480681
 Enzo Fernández Argentina        7.5         7.226126
      Luis Díaz  Colombia        8.1         6.899292
Youri Tielemans   Belgium        6.1         6.624349
   Lamine Yamal     Spain       10.0         6.573912

--- FWD ---
             name        team  price_raw  expected_points
    Kylian Mbappé      France       10.5        12.773151
     Lionel Messi   Argentina       10.0        11.781866
  Mikel Oyarzabal       Spain        8.1         8.219121
 Lautaro Martínez   Argentina        8.8         7.642593
     Breel Embolo Switzerland        7.5         7.389308
    Romelu Lukaku     Belgium        7.4         7.294657
       Harry Kane     England       10.5         6.938898
   Erling Haaland      Norway       10.5         6.728981
   Jonathan David      Canada        7.0         6.400784
Cristiano Ronaldo    Portugal       10.0         6.259553
```

In [131…

```python
player1 = "Dayot Upamecano"
player2 = "Achraf Hakimi"

cols = [
    "name", "team", "position", "price_raw", "expected_points",
    "e_appearance", "e_goal", "e_assist", "e_cs", "e_gc",
    "e_saves", "e_pen_save", "e_tackles", "e_cc", "e_sot",
    "e_yc", "e_rc", "e_og", "e_pw", "e_pc",
    "e_quali", "e_scouting"
]

comparison = df.loc[
    df["name"].isin([player1, player2]),
    cols
].set_index("name").T

print(comparison)
```

name	Dayot Upamecano	Achraf Hakimi
team	France	Morocco
position	DEF	DEF
price_raw	5.3	6.0
expected_points	7.483832	7.741311
e_appearance	1.781801	1.870466
e_goal	1.128603	2.442752
e_assist	0.212127	0.560888
e_cs	2.799856	1.837353
e_gc	-0.105847	-0.249934
e_saves	0.0	0.0
e_pen_save	0.0	0.0
e_tackles	0.0	0.0
e_cc	0.0	0.0
e_sot	0.0	0.0
e_yc	-0.04441	-0.10975
e_rc	-0.015178	-0.025121
e_og	-0.006442	-0.012225
e_pw	0.051996	0.067138
e_pc	-0.006663	-0.00976
e_quali	1.687989	1.369504
e_scouting	0.0	0.0

Results of the final run before closing 18:00 time: